



Qualcomm Technologies, Inc.

Qualcomm Linux Kernel Guide

80-70014-3 Rev. AR

July 15, 2024

Contents

1 Overview	7
2 Getting started	8
2.1 Access Qualcomm Linux kernel source code	8
2.2 Identify supported Qualcomm machines	9
2.3 Build the Yocto image recipes and kernel configurations	9
2.4 Access the platform device tree	11
2.5 View out-of-tree DT overlay fragments	12
2.6 View out-of-tree kernel modules	13
2.7 Build device image	13
2.8 Bring up the device	14
3 Features	16
3.1 Boot	16
3.2 Platform support	17
3.2.1 QCM6490 device tree	18
3.2.2 Kernel configurations	19
3.3 Remoteproc subsystem	20
3.3.1 Qualcomm remoteproc support	20
3.3.2 Remoteproc recovery and subsystem restart	21
3.3.3 Coredump	22
3.3.4 Remoteproc configuration and firmware	22
3.4 Memory	23
3.5 Scheduler	27
3.5.1 CPU topology and EAS	27
3.5.2 Per entity load tracking	28
3.5.3 SchedUtil governor	28
3.5.4 Utilization clamping (UCLAMP)	28
3.6 Dynamic voltage and frequency scaling (DVFS)	28
3.6.1 CPU DVFS	29
3.6.2 Cache and memory DVFS	29

3.7 Virtualization	29
3.7.1 Qualcomm virtualization solution	30
3.7.2 Guest VM build support	32
3.7.3 Launch guest virtual machine	33
3.8 Real-time (RT) kernel overview	44
4 Yocto support	50
4.1 Kernel source	50
4.2 Kernel recipe	50
4.3 Unpack kernel source	51
4.4 Make kernel changes	51
4.5 Commit kernel changes	52
4.6 Build kernel image	52
4.7 Maintain kernel changes	52
4.8 Clean up the workspace	53
5 Customize	54
5.1 Kernel development	54
5.2 Kernel configurations	55
5.3 Debug builds	55
5.4 Update kernel command line	55
5.5 Platform support	56
5.6 Update ESP with images	58
5.7 Customize initramfs	58
5.8 Add kernel module	59
5.9 Disable console log	60
5.10 Pinctrl configuration	60
5.11 General GPIO usage	62
5.12 ZRAM as swap	64
5.13 Extend memory map	64
5.14 Add custom CMA heaps	66
5.15 Scheduler	67
5.16 Change default CPU frequency governor	68
5.17 Customize cache and memory DVFS	68
5.18 Post boot settings	69
6 Debug	73
6.1 Enable SSH	75
6.2 Retrieve information from device	75
6.3 Enable debugfs	75

6.4 Debugging kernel and modules through KGDB	76
6.5 Function trace	78
6.5.1 Enable memory-mapped input/output traces	78
6.5.2 Kprobes	79
6.6 Troubleshoot kernel problems	80
7 References	82

Tables

Table 1-1: Supported chipsets.....	7
Table 2-1: Qualcomm Linux supported image recipes.....	9
Table 2-2: Kernel configuration fragments.....	10
Table 2-3: Kernel build configuration type and configurations.....	10
Table 2-4: Qualcomm device tree source.....	11
Table 2-5: Kernel build artifacts.....	14
Table 3-1: CPU topology and capacity.....	27
Table 3-2: CPU DVFS variables.....	29
Table 3-3: Features of Gunyah.....	31
Table 3-4: Supported parameters.....	34
Table 5-1: Pinctrl data objects.....	60
Table 5-2: Syntax for carved out region.....	65
Table 5-3: Syntax for adding CMA region.....	66
Table 5-4: Supported fields for [Unit] section.....	70
Table 5-5: Supported fields for [Service] section.....	70
Table 6-1: List of kernel commands.....	75
Table 6-2: Reasons of device crashes.....	79

Figures

Figure 1-1: Qualcomm Linux kernel workflow..... 7

Figure 3-1: Device tree overlay..... 18

Figure 3-2: Remoteproc support..... 21

Figure 3-3: Remoteproc crash sequence.....21

Figure 3-4: Virtualization..... 30

Figure 3-5: Qualcomm virtualization solution architecture..... 31

Figure 3-6: Qualcomm virtualization solution.....32

Figure 3-7: Build sequence.....44

Figure 3-8: RT kernel verification..... 46

1 Overview

Qualcomm® Linux® facilitates porting and customizing the Linux kernel to devices with Qualcomm chipsets. It uses a long-term support Linux kernel (6.6.x) and the GNU compiler collection (GCC) toolchain that is compatible with the Yocto project.

The kernel is integrated and aligned to support Qualcomm Linux through Qualcomm supported meta layers. Qualcomm Linux provides a way to build and customize the kernel and other packages to define a distribution according to the needs of a device or application.

The following information enables you to understand the procedures to:

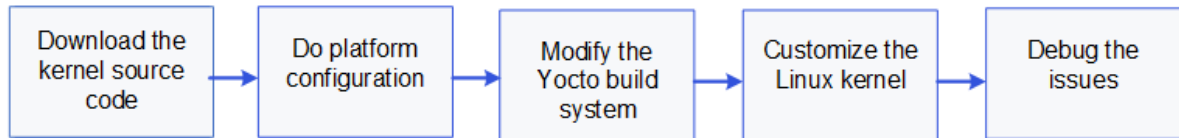


Figure 1-1 Qualcomm Linux kernel workflow

The Qualcomm Linux distribution supports the following chipsets and development kits:

Table 1-1 Supported chipsets

Chipset	Supported device
QCM6490/QCS6490	■ QCS6490 RB3 Gen2 core development kit ■ QCS6490 RB3 Gen2 optional mezzanine boards
QCM5430/QCS5430	QCS5430 RB3 Gen2 core development kit

For more information on QCS6490 optional mezzanines, see [Qualcomm RB3 Gen 2 Development Kit Guide](#).

2 Getting started

This information explains how to download, configure, and develop the Qualcomm Linux kernel for Qualcomm platforms and their associated components to build and flash system images.

Before you begin, set up your host machine as described in the [Qualcomm Linux Build Guide](#). This guide also provides information about the common Qualcomm Linux workflows.

2.1 Access Qualcomm Linux kernel source code

To access the source code for the Qualcomm Linux kernel, see the [Qualcomm Linux Build Guide](#) and get all necessary meta layers.

Qualcomm Linux uses supports the 6.6 LTS kernel, which is available through the kernel recipe under the meta-qcom-hwe layer in the `recipes-kernel/linux/linux-kernel-qcom_6.6.bb` file.

The Linux kernel recipe in Qualcomm Linux is called as `linux-kernel-qcom`. See the `recipes-kernel/linux/linux-kernel-qcom_6.6.bb` file to know the Git uniform resource identifier (URI) used for kernel source code download.

For more information on Qualcomm Linux layers, see [Qualcomm BSP metadata layers](#).

NOTE Yocto recipe refers to Qualcomm kernel sources that are publicly hosted at [Codelinaro](#).

The following example shows the Qualcomm Linux kernel recipe:

```
SECTION = "kernel"

SUMMARY = "Linux kernel for QCOM devices"
DESCRIPTION = "Recipe to build Linux kernel"

LICENSE = "GPLv2.0-with-linux-syscall-note"
LIC_FILES_CHKSUM = "file://COPYING;md5=6bc538ed5bd9a7fc9398086aedcd7e46"

inherit kernel

COMPATIBLE_MACHINE = "(qcom)"

SRC_URI += "git://git.codelinaro.org/clo/la/kernel/
qcom.git;protocol=https;rev=aea6160f0ba1c279fcdeded51e68a0da090f5ed9;branch=ke
rnel.qclinux.1.0.r1-rel \
        ${@bb.utils.contains('DISTRO_FEATURES', 'selinux', ' file://
selinux.cfg', '', d)} \
```

```

    ${@bb.utils.contains('DISTRO_FEATURES', 'selinux', ' file://
selinux_debug.cfg', '', d)} \"/>

```

2.2 Identify supported Qualcomm machines

The Qualcomm machine configuration files are present in the `meta-qcom-hwe/conf/machine/` directory.

The following example shows the list of supported machines that can be configured for Yocto machine configuration:

```
$ ls -l meta-qcom-hwe/conf/machine/qcm6490.conf
```

The following example shows the machine configuration and the kernel selection for QCM6490:

```
$ less meta-qcom-hwe/conf/machine/qcm6490.conf
```

```

SOC_FAMILY:prepend = "qcom:"
require conf/machine/include/soc-family.inc

```

```
PREFERRED_PROVIDER_virtual/kernel = "linux-kernel-qcom"
```

QCM6490 selects the `linux-kernel-qcom` kernel recipe to compile the kernel.

NOTE QCM5430 is a variant of QCM6490 and uses the same configurations as QCM6490. For more information about the QCM5430 device tree, see [Access the platform device tree](#).

2.3 Build the Yocto image recipes and kernel configurations

You can access the Qualcomm Linux image recipes to modify the kernel configurations. The following table lists the supported `meta-qcom-hwe` images:

Table 2-1 Qualcomm Linux supported image recipes

Image name	Description
<code>qcom-minimal-image</code>	This is a minimal rootfs image that boots to the shell.
<code>qcom-console-image</code>	This is a boot to shell image with the package group to bring in all basic packages.
<code>qcom-multimedia-image</code>	This image recipe includes recipes for multimedia software components, such as, audio, Bluetooth®, camera, computer vision, display, and video.
<code>qcom-multimedia-test-image</code>	This image recipe includes tests.

After downloading the Qualcomm Linux code, use the `bitbake` command to build the image.

To build the images listed in the [Table 2-1](#), the Qualcomm Linux kernel recipe uses one of the following kernel configuration fragments:

Table 2-2 Kernel configuration fragments

Kernel configuration fragments	Description
<code><kernel_src>/arch/arm64/configs/qcom_defconfig</code>	The base configuration of the product must be aligned with the upstream kernel.
<code><kernel_src>/arch/arm64/configs/qcom_debug.config</code>	Debug configuration fragment from the upstream kernel.
<code><kernel_src>/arch/arm64/configs/qcom_addons.config</code>	Additional Qualcomm downstream additions on top of the upstream aligned base.
<code><kernel_src>/arch/arm64/configs/qcom_addons_debug.config</code>	Qualcomm downstream debug enablement.

To build Qualcomm Linux including kernel, run the following commands:

```
$ MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment
```

```
# build qcom linux console image
$ bitbake qcom-console-image
```

```
# Build Images are produced under build-qcom-wayland/tmp-glibc/deploy/images/
qcm6490/qcom-console-image/
```

The kernel default builds as defined in the `linux-kernel-qcom_6.6.bb` Linux kernel recipe, supports #3 (perf) and #4 (debug) builds mentioned in the following table. The default build configuration is #3 (perf).

Table 2-3 Kernel build configuration type and configurations

Serial number	Kernel build configuration type	Defconfig/config fragments
1	Base	<code>arch/arm64/configs/qcom_defconfig</code>
2	Debug enabled base	■ <code>arch/arm64/configs/qcom_defconfig</code> ■ <code>arch/arm64/configs/qcom_debug.config</code>
3	Base with downstream additions	■ <code>arch/arm64/configs/qcom_defconfig</code> ■ <code>arch/arm64/configs/qcom_addons.config</code>
4	Debug enabled base with downstream	■ <code>arch/arm64/configs/qcom_defconfig</code> ■ <code>arch/arm64/configs/qcom_debug.config</code> ■ <code>arch/arm64/configs/qcom_addons.config</code> ■ <code>arch/arm64/configs/qcom_addons_debug.config</code>

For more information on configuring the kernel, see [Kernel configurations](#).

Qualcomm debug build

Use the following command to compile a debug build:

```
$ DEBUG_BUILD=1 bitbake qcom-console-image
```

To modify the build configurations update the `KERNEL_DEFCONFIG` and `KERNEL_CONFIG_FRAGMENTS` variables in the `linux-kernel-qcom_6.6.bb` kernel recipe in the `meta-qcom-hwe` layer.

Example kernel configuration

The following example shows the kernel configuration using the `linux-kernel-qcom_6.6.bb` kernel recipe:

```
KERNEL_DEFCONFIG = "${S}/arch/arm64/configs/qcom_defconfig"
```

```
KERNEL_CONFIG_FRAGMENTS:append = " ${S}/arch/arm64/configs/qcom_addons.config"
KERNEL_CONFIG_FRAGMENTS:append = " ${@oe.utils.vartrue('DEBUG_BUILD', '${S}/arch/arm64/configs/qcom_debug.config', '', d)}"
KERNEL_CONFIG_FRAGMENTS:append = " ${@oe.utils.vartrue('DEBUG_BUILD', '${S}/arch/arm64/configs/qcom_addons_debug.config', '', d)}"
```

For more information on the build instructions, see [Build with QSC CLI](#) in the [Qualcomm Linux Build Guide](#).

2.4 Access the platform device tree

The Qualcomm device tree source inclusion (DTSI) and device tree source (DTS) files specific to supported development kits are located under the kernel source in the `arch/arm64/boot/dts/qcom/` directory. These Qualcomm files segregate the upstream-aligned and Qualcomm downstream additions.

The following table lists the Qualcomm device tree files for the RB3 Gen 2 development kit. By default the addons device tree blob (DTB) is used.

Table 2-4 Qualcomm device tree source

Device tree source	Details
<code>arch/arm64/boot/dts/qcom/sc7280.dtsi</code>	QCS6490 SoC is derived from SC7280.
<code>arch/arm64/boot/dts/qcom/qcm6490.dtsi</code>	QCS6490 DTSI that hosts any changes are different from the SC7280 SoC.
<code>arch/arm64/boot/dts/qcom/qcm6490-addons.dtsi</code>	QCS6490 SoC specific downstream additions.
<code>arch/arm64/boot/dts/qcom/qcs6490-rb3gen2.dts</code>	QCS6490 RB3 Gen 2 development kit device tree source.

Table 2-4 Qualcomm device tree source (cont.)

Device tree source	Details
arch/arm64/boot/dts/qcom/qcs6490-addons-rb3gen2.dts	QCS6490 RB3 Gen 2 development kit specific downstream additions.
arch/arm64/boot/dts/qcom/qcs6490-addons-rb3gen2.dts	QCS5430 feature pack-1 RB3 Gen 2 development kit specific downstream additions.
arch/arm64/boot/dts/qcom/qcs5430-fp2-addons-rb3gen2.dts	QCS5430 feature pack-2 RB3 Gen 2 development kit downstream additions.

The Qualcomm Linux machine configuration `conf/machine/qcm6490.conf` file has the configuration-related data for the necessary DTB selection.

The following example shows the DTB inclusion into the device configuration file:

```
KERNEL_DEVICETREE = " \
                    qcom/qcs6490-addons-rb3gen2.dtb \
                    "
```

2.5 View out-of-tree DT overlay fragments

QCM6490 machine configuration `conf/machine/qcm6490.conf` file includes additional out-of-tree DTB overlays (DTBOs).

Graphics, camera, wireless local area network (WLAN) drivers, and their device tree configurations are maintained out of the kernel source tree. Their respective device tree source overlay (DTSO) fragments are overlaid at the build time onto the base DTBs.

The DTBO configuration is done in the `conf/machine/qcm6490.conf` file.

The following example shows the DTBO configuration:

```
# OUT_OF_KERNEL_DTBO - qcm6490.conf
# Additional list of DTBOs to be overlaid on top of base kernel devicetree
files
# Format - KERNEL_TECH_DTBOs[<base-dtb-name>] = "<dtbo1 <dtbo2> ..."
# For example:

KERNEL_TECH_DTBOs[qcs6490-addons-rb3gen2] = "qcm6490-graphics.dtbo qcm6490-
wlan-rb3gen2.dtbo qcm6490-display-rb3gen2.dtbo qcm6490-bt.dtbo"
```

2.6 View out-of-tree kernel modules

Most of the kernel drivers are compiled from upstream kernel sources. However, some drivers are maintained outside the kernel source tree and are built as kernel modules using the Yocto build system.

In the following example, Bluetooth is an out-of-tree kernel module with the recipe located at `recipes-connectivity/bt_dlm_kernel/bt-dlm-kernel_git.bb` that compiles the driver as a kernel module. To auto-load the kernel module on boot, you can add the module name to the `KERNEL_MODULE_AUTOLOAD` variable.

```
# Example out-of-tree Kernel module recipe
DESCRIPTION = "QCOM BT drivers"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/${
{LICENSE}};md5=801f80980d171dd6425610833a22dbe6"

inherit module

SRC_URI += "git://git.codellinaro.org/clo/le/platform/vendor/qcom-
opensource/bt-
kernel.git;protocol=https;rev=9e5cf29625d60f78e88440c94d096a9139445c00;branch=
bt-performant.qclinux.1.0.r1-rel \
        file://bt_dlm \
        file://bt_dlm.service \
        "

S = "${WORKDIR}/git"

RPROVIDES:${PN} += "kernel-module-bt-kernel"

EXTRA_OEMAKE += "MACHINE='${MACHINE}'"
MAKE_TARGETS = "modules"
MODULES_INSTALL_TARGET = "modules_install"
# Kernel module to be autoloading
KERNEL_MODULE_AUTOLOAD += "bt_fm_slim"
```

2.7 Build device image

Use the following Yocto commands to build the full image and kernel:

```
# setup the build environment
$ export SHELL=/bin/bash

$ source setup-environment
```

```
# (select "qcm6490 meta-qcom" → Enter → select "qcom-wayland meta-qcom-
distro" → Enter)

# build qcom linux console image
$ bitbake qcom-console-image

# Build kernel only
$ bitbake linux-kernel-qcom

# Build Images are produced under
# build-qcom-wayland/tmp-glibc/ deploy/images/qcm6490/qcom-console-image/
```

For more information on how to build different images for QCM6490 supported Qualcomm distribution, see [Build the Yocto image recipes and kernel configurations](#).

2.8 Bring up the device

The `efi.bin` file contains the systemd-boot (boot manager), kernel, DTB, and initramfs images. The Yocto build packs these images into the `efi.bin` that can be flashed to the EFI system partition (ESP) to boot the device.

The following device tree blobs are used by default to build the `efi.bin` file:

- `qcs6490-addons-rb3gen2.dtb`

Device tree blobs are updated or added to the `KERNEL_DEVICETREE` variable in the `conf/machine/qcm6490.conf` file.

The `efi.bin` image can be flashed using the following `fastboot` command:

```
# Bring the device in fastboot mode
# cd to the source root location
$ cd build-qcom-wayland/tmp-glibc/ deploy/images/qcm6490/qcom-console-image
$ fastboot flash efi efi.bin
$ fastboot reboot
```

After the Qualcomm Linux build, the kernel build is completed, the kernel-related images are created in the `tmp-glibc/ deploy/images/qcm6490/` directory.

The following table lists the build artifacts related to the kernel:

Table 2-5 Kernel build artifacts

Image	Image name	Build deploy path	Details
Kernel executable and linking format (ELF)	<code>vmlinux</code>	<code>temp-glibc/ deploy/images/qcm6490/</code>	Output kernel ELF with debug symbols.
Initramfs	<code>initramfs-qcom-image-qcm6490.cpio.gz</code>	<code>tmp-glibc/ deploy/images/qcm6490/</code>	Initramfs in copy in, copy out (CPIO) file format.

Table 2-5 Kernel build artifacts (cont.)

Image	Image name	Build deploy path	Details
Kernel image	Image	tmp-glibc/deploy/ images/qcm6490/	Kernel raw image binary, systemd-boot does not support compressed images.
Kernel modules	Different kernel dynamically loadable kernel modules (DLKMs) modules-qcm6490.tgz	tmp-glibc/deploy/ images/qcm6490/	Kernel drivers modules.
Device tree blobs	qcm6490-addons-rb3gen2.dtb	tmp-glibc/deploy/ images/qcm6490/	Individual device tree blobs.
ESP partition	efi.bin	tmp-glibc/deploy/ images/qcm6490/	All required boot images including systemd-boot, kernel, DTB, and initramfs are packaged into the extensible firmware interface (efi.bin) binary.

3 Features

The Qualcomm Linux kernel offers various features to run an upstream Linux kernel on Qualcomm devices.

Here are some key features and advancements related to running the mainline Linux kernel on the Qualcomm platforms:

- Qualcomm Linux BSP tailored to support devices using Qualcomm chipsets.
- Device tree corresponds to Qualcomm development kits.
- Multiple different build configurations to facilitate the use of Qualcomm chipsets.
- The Linux kernel integrated to the Yocto build system.
- The Qualcomm Linux kernel is aligned to kernel community standards. Qualcomm specific additions are maintained separately.
- Support to configure, customize, and kernel build images that can be flashed and booted on devices using Qualcomm chipsets.
- The Qualcomm Linux kernel is part of an EFI image, using the Qualcomm unified extensible firmware interface (UEFI) and standard systemd-boot boot manager.
- Downstream Qualcomm infrastructure supports are maintained to provision custom use cases like RAM dump and secure use cases.
- Compatible with standard Linux kernel debug methods using tracing and Ram dump.
- Virtualization support for untrusted virtual machines using the Gunyah™ Hypervisor Software.
- Periodic long-term support (LTS) merges from the corresponding LTS kernel branch to get the latest security and stability fixes.

3.1 Boot

Qualcomm Linux supports systemd-boot as the UEFI boot manager to load and boot the Linux kernel. The Linux kernel in this case is built as an EFI stub.

Boot flow and architecture

Cold boot begins the execution from the primary boot loader (PBL) that sets up the initial system for the eXtensible boot loader (XBL). The XBL performs wider system initialization including DDR initialization and loads a UEFI image. UEFI provides a rich firmware interface that loads systemd-boot as a boot manager on Qualcomm Linux to manage the OS images.

For more information on complete cold boot flow, see [Cold boot architecture](#) in the [Qualcomm Linux Boot Guide](#).

Systemd-boot

Systemd-boot is a UEFI boot manager that executes EFI images, provides boot entries, and supports unified kernel images. Systemd-boot supports the following:

- Boot entries: The type 1 boot loader specification entries are in the `loader/entries/` directory on the ESP. These files describe Linux kernel images with the associated initrd images, and other EFI executables.
- Unified kernel images (UKI): The type 2 type 2 boot loader specification EFI unified kernel images are executable EFI binaries in the `/EFI/Linux/` directory on the ESP.

For information on the boot entries and unified kernel images, see [BootLoader Specification](#).

Systemd-boot is a part of the systemd package that is a part of the meta Yocto layer. The Yocto recipe builds systemd-boot using the `systemd-boot_254.4.bb` recipe file, and the `uki.bbclass` recipe file handles the ESP image generation.

For more information on systemd-boot, see [systemd-boot](#).

Linux kernel as EFI stub

The EFI boot stub enables booting the Linux kernel directly without a conventional EFI boot loader.

The boot firmware can load the EFI image as an executable file when the Linux kernel is compiled with the `CONFIG_EFI_STUB` kernel configuration option. In this case, the firmware loader jumps to the `EFI boot stub` location in the EFI image `drivers/firmware/efi/libstub/`, which then boots the kernel.

For Arm® (Arm64), where there is no compressed kernel support, the kernel image acts as a portable executable (PE) file format or common object file format (COFF) image and the EFI stub is linked into the kernel.

For more information on booting the Linux kernel as an EFI image, see [The EFI Boot Stub](#).

Boot images and ESP/boot partition

The ESP serves as a storage location for `efi.bin` image that packages systemd-boot, UKI, and DTBs. The UKI includes kernel image, initramfs, and kernel command line arguments.

The UEFI firmware launches the UEFI boot loader and loads the kernel boot images. The ESP is formatted with the file allocation table (FAT) file system supported in the UEFI specification. For more information on UKI image format and corresponding support in base meta layers, see [systemd-boot](#).

DTB selection

All the device tree blobs are packaged as part of `efi.bin`. UEFI selects and loads the right DTB for the Linux kernel.

To pack the DTB for boot time selection, see [Platform support](#).

3.2 Platform support

Qualcomm Linux uses the following procedures to choose device tree files, make changes to kernel configuration, and build out-of-tree kernel modules.

3.2.1 QCM6490 device tree

Qualcomm Linux supports a device tree overlay feature that supports the maintenance and merging of out-of-device tree blobs with the baseline device tree blobs.

QCM6490 system-on-chip (SoC) device tree support is in the kernel source in the `arch/arm64/boot/dts/qcom` directory.

Table 2-4 lists how the device tree maintains a clear separation to contain downstream additions provisioning the upstream aligned base distinctly:

There is downstream content that is hosted outside the kernel source. The out-of-tree drivers also maintain their device tree overlay additions outside the kernel. During the build process, the corresponding device tree overlay is merged with the base board DTB and a unified DTB is generated, which is used with the boot images.

The following figure shows the build process for device tree overlay, and DTB generation:

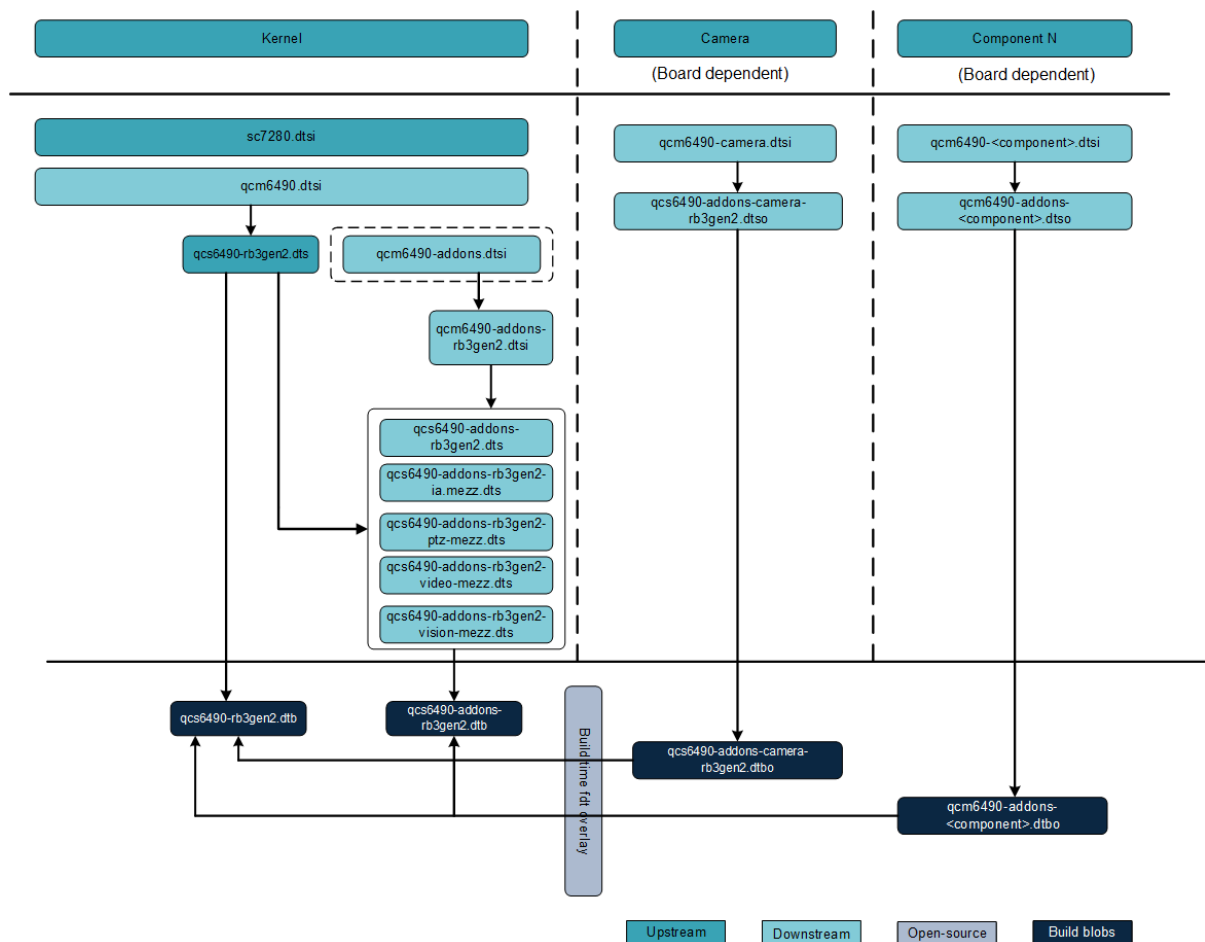


Figure 3-1 Device tree overlay

The build system uses a `fdt_overlay` tool to provision the out-of-tree device tree overlay handling during the build.

The DTBO merge supports merging the DTBO files listed in the `meta-qcom-hwe/conf/machine/qcm6490.conf` file.

The device tree variables are enumerated as follows:

```
# List of dtbs for corresponding supported qcm6490 platforms
KERNEL_DEVICETREE = " \
                    qcom/qcs6490-addons-rb3gen2.dtb \
                    "

# Additional list of dtbos to be overlaid on top of base kernel devicetree
files
# Format - KERNEL_TECH_DTBOS[<base-dtb-name>] = "<dtbo1 <dtbo2> ..."
# For example:

KERNEL_TECH_DTBOS[qcs6490-addons-rb3gen2] = "qcm6490-graphics.dtbo qcm6490-
wlan-rb3gen2.dtbo qcm6490-display-rb3gen2.dtbo qcm6490-bt.dtbo"
```

DTBO merge sequence

The `merge_dtbos()` task in the `image-qcom-deploy.bbclass` class merges DTBOs using the `fdtoverlay` tool.

The `merge_dtbos()` task iteration occurs in the following sequence:

1. The task starts with the DTBO filenames listed in the configuration file.
2. It then iterates through the `KERNEL_DEVICETREE` variable.
3. For each base DTB, it overlays the DTBOs listed in the `KERNEL_TECH_DTBOS` variable.
4. After the overlay process is completed, the final DTB output is generated.

3.2.2 Kernel configurations

The kernel recipe `linux-kernel-qcom_6.6.bb` uses the following configuration fragments to build the images.

```
KERNEL_DEFCONFIG = "${S}/arch/arm64/configs/qcom_defconfig"

KERNEL_CONFIG_FRAGMENTS:append = " ${S}/arch/arm64/configs/qcom_addons.config"
KERNEL_CONFIG_FRAGMENTS:append = " ${@oe.utils.vartrue('DEBUG_BUILD', '${S}/
arch/arm64/configs/qcom_debug.config', '', d)}"
KERNEL_CONFIG_FRAGMENTS:append = " ${@oe.utils.vartrue('DEBUG_BUILD', '${S}/
arch/arm64/configs/qcom_addons_debug.config', '', d)}"
```

Use the following commands to generate the debug build using debug configuration fragments:

```
# Set DEBUG_BUILD to 1 to compile debug enabled build

$ source setup-environment
$ DEBUG_BUILD=1 bitbake linux-kernel-qcom
```

For more information on different configuration fragments, see [Getting started](#).

3.3 Remoteproc subsystem

Remoteproc framework is used to load firmware into other subsystems such as the audio digital signal processor (aDSP) and compute digital signal processor (cDSP) on Qualcomm platforms.

The remoteproc framework is divided into two parts— a core framework and remoteproc drivers. The core framework contains common logic and the drivers contain platform-specific operations to manage the corresponding cores. A remoteproc driver registers a remoteproc instance and a set of operations with the core framework. The core framework loads the firmware and starts or stops the remote processor.

There are several heterogeneous remote processors present on an SoC in an asymmetric multiprocessing (ASMP) configuration. The remote processors run different instances of firmware (or OS).

For more information on the standard framework that Qualcomm Linux uses to manage other cores, see [Remote Processor Framework](#).

3.3.1 Qualcomm remoteproc support

Qualcomm has enhanced the remoteproc framework for peripheral firmware authentication using the upstream peripheral authentication service (PAS) driver.

Remoteproc driver in the Qualcomm Linux implements a peripheral authentication service (PAS) driver, which is a TrustZone based peripheral image loader for remote processors on Qualcomm SoC devices.

For more information, see [qcom_q6v5_pas.c](#).

The remote processors are used to manage the lifecycle of various co-processors, such as aDSP, cDSP, modem peripheral subsystem (MPSS), and wireless processor subsystem (WPSS). The following is the list of supported co-processors on QCM6490-based boards:

- RB3 Gen 2: aDSP, cDSP, and WPSS

User interface: The remoteproc framework tracks registered remoteproc devices and provides a user interface to boot and shutdown the devices. User space can query and change the current state of a remoteproc using the following `sysfs` interface:

```
remoteproc user space interface
# start remoteproc:
$ echo "start" > /sys/class/remoteproc/remoteprocN/state

# stop remoteproc:
$ echo "stop" > /sys/class/remoteproc/remoteprocN/state
```

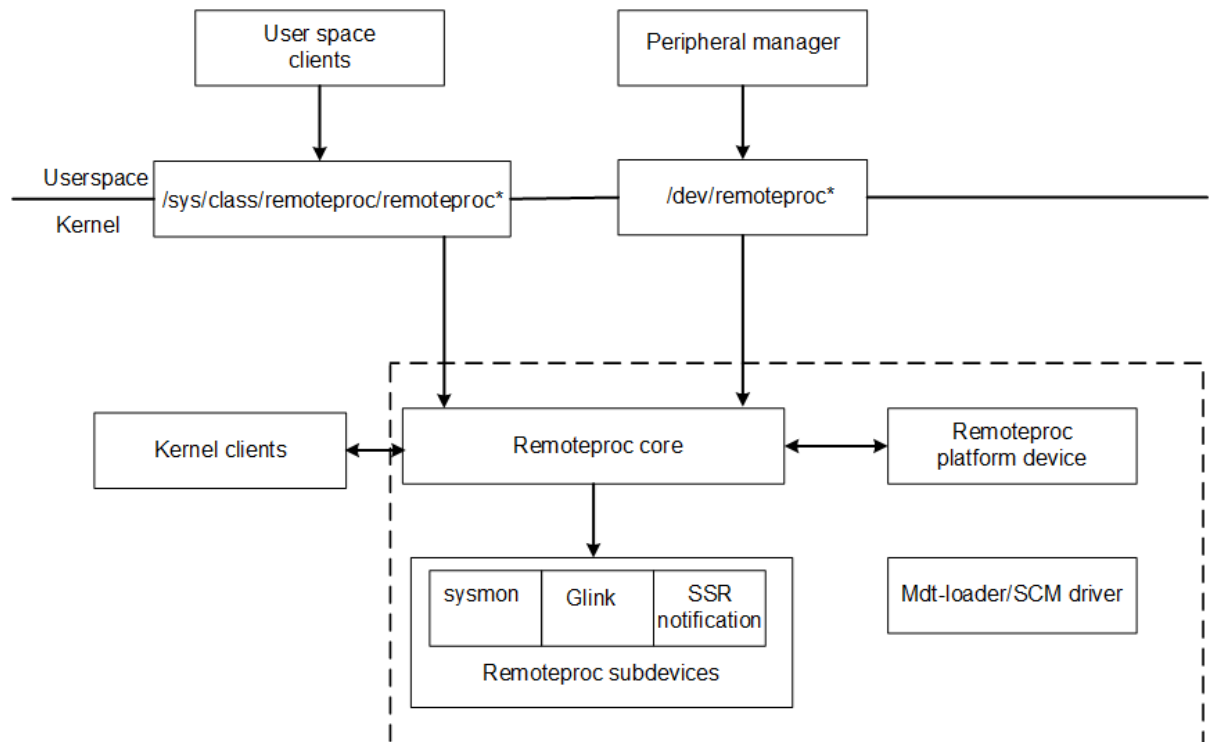


Figure 3-2 Remoteproc support

3.3.2 Remoteproc recovery and subsystem restart

The remoteproc subdevices manage communication contexts for remote processor subsystem crash and recovery scenarios.

Entities such as communication contexts must determine when a remote processor is started or shut down. Each subdevice has two operations,

- `probe()` called after the remoteproc `start()`
- `remove()` called before `stop()`

When a fatal error occurs on a remote processor, the remoteproc driver handles it and invokes `rproc_report_crash()`, which triggers recovery of the faulty remote processor.

The recovery handler does the following:

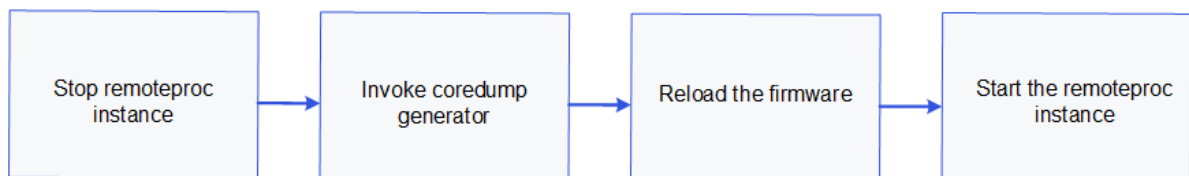


Figure 3-3 Remoteproc crash sequence

To enable and disable recovery using the recovery `debugfs` node from the user space:

```
# remoteproc user space interface
```

```
# Disable SSR:
# echo disabled > /sys/kernel/debug/remoteproc/remoteprocN/recovery
```

3.3.3 Coredump

Remote processor coredumps are supported for debugging the subsystem crash issues using the Linux kernel devcoredump feature.

When a subsystem crash happens, the devcoredump exposes a snapshot of the memory of recovering remoteproc in the `/sys/class/devcoredump/devcdN/data` node, and an associated crash uevent is sent to the user space. When the sysfs node is read, it provides the segments in an ELF container, and a write frees up the resources and destroys the devcoredump instance.

A debugfs node is exposed to enable and disable coredump from the user space.

```
# Disable coredumps:
$ echo disabled > /sys/kernel/debug/remoteproc/remoteprocN/coredump
```

NOTE QCM6490 platform implements kernel panic on remoteproc crash (fatal error on remote processor) to collect the full RAM dump for crash analysis and further debug. The kernel panic is enabled only when the coredump is disabled.

3.3.4 Remoteproc configuration and firmware

The remoteproc devices are configured in the respective device tree files using DT bindings. The configuration includes compatible device names, memory regions, interrupt settings, and clock references according to the DT bindings in the `Documentation/devicetree/bindings/remoteproc/` directory.

The following example shows the QCM6490 aDSP remoteproc device tree configuration in the `arch/arm64/boot/dts/qcom/qcm6490.dtsi` file:

```
remoteproc_adsp: remoteproc@3000000 {
    compatible = "qcom,sc7280-adsp-pas";
    reg = <0x0 0x03000000 0x0 0x100>;

    interrupts-extended = <&pdc 6 IRQ_TYPE_EDGE_RISING>,
                          <&adsp_smp2p_in 0
IRQ_TYPE_EDGE_RISING>,
                          <&adsp_smp2p_in 1
IRQ_TYPE_EDGE_RISING>,
                          <&adsp_smp2p_in 2
IRQ_TYPE_EDGE_RISING>,
                          <&adsp_smp2p_in 3
IRQ_TYPE_EDGE_RISING>,
                          <&adsp_smp2p_in 7
IRQ_TYPE_EDGE_RISING>;
    interrupt-names = "wdog", "fatal", "ready", "handover",
                      "stop-ack", "shutdown-ack";
```

```

        clocks = <&rpmhcc RPMH_CXO_CLK>;
        clock-names = "xo";
        <snip>
    };

```

All firmware files are present in the `/lib/firmware` directory in `rootfs` and the related configurations are done in the corresponding board device tree `arch/arm64/boot/dts/qcom/qcs6490-rb3gen2.dts` file.

The following example shows the remoteproc DT configuration:

```

&remoteproc_adsp {
    firmware-name = "qcom/qcm6490/adsp.mdt";
    status = "okay";
};

```

Sample logs

The following log is displayed when a remoteproc boots successfully:

```

# remoteproc success
"remoteproc remoteproc1: remote processor 4080000.remoteproc is now up"

```

The following log is displayed when a remoteproc firmware load fails:

```

# remoteproc failure
Error log: "remoteproc remoteproc0: Direct firmware load for qcom/qcm6490/modem.mdt failed with error -2"

# Caused by: error loading firmware.
# Solution: ensure that the modem firmware is copied to /lib/firmware/qcom/qcm6490 in rootfs.

```

3.4 Memory

Qualcomm Linux kernel baseline supports all memory management features and allocators. The following information outlines customizing the memory map and performing heap management.

For more information on Linux kernel memory management, see [Memory management](#).

Platform memory map

Platform memory map describes the areas that are reserved for subsystems, such as modem, camera, aDSP, and cDSP during kernel boot.

The memory map sets `no-map` for the carved out regions in the DTBI making them inaccessible to the kernel.

Carved out regions are configurable and are defined in the `arch/arm64/boot/dts/qcom/qcm6490.dtsi` DTSI file in the `reserved-memory` node.

```
reserved-memory {
    cdsp_secure_heap_mem: cdsp-secure-heap@81800000 {
        reg = <0x0 0x81800000 0x0 0x1e00000>;
        no-map;
    };

    camera_mem: camera@84300000 {
        reg = <0x0 0x84300000 0x0 0x500000>;
        no-map;
    };

    wpss_mem: wpss@0x84800000 {
        reg = <0x0 0x84800000 0x0 0x1900000>;
        no-map;
    };

    adsp_mem: adsp@86100000 {
        reg = <0x0 0x86100000 0x0 0x2800000>;
        no-map;
    };
};
```

The following early boot log shows carved out region creation for different subsystems:

```
[    0.000000] OF: reserved mem: 0x0000000081800000..0x00000000835fffff
(30720 KiB) nomap non-reusable cdsp-secure-heap@81800000
[    0.000000] OF: reserved mem: 0x0000000084300000..0x00000000847fffff (5120
KiB) nomap non-reusable camera@84300000
[    0.000000] OF: reserved mem: 0x0000000084800000..0x00000000860fffff
(25600 KiB) nomap non-reusable wpss@0x84800000
[    0.000000] OF: reserved mem: 0x0000000086100000..0x00000000888fffff
(40960 KiB) nomap non-reusable adsp@86100000
```

Contiguous memory allocator (CMA)

Qualcomm Linux distribution supports CMA to allocate large physically contiguous memory. CMA reserves a large physically contiguous memory area at boot time and provides physically continuous memory to CMA allocation. When not in use, CMA memory is available to the kernel buddy allocator for movable allocations.

To modify the size of the default CMA region, you can pass `cma=size_in_MB` in the kernel command line arguments. For more information on how to use the kernel parameter, see the following example:

```
cma=nn[MG]@[start[MG] [-end[MG]]]
[KNL,CMA]
Sets the size of kernel global memory area for
```

contiguous memory allocations and optionally the placement constraint by the physical address range of memory allocations. A value of 0 disables CMA altogether. For more information, see `kernel/dma/contiguous.c`

The custom CMA regions are defined under the `reserved-memory` node with a `shared-dma-pool` compatible tag indicating the CMA region:

```
adsp_heap_mem: adsp-heap {
    compatible = "shared-dma-pool";
    alloc-ranges = <0x0 0x00000000 0x0 0xffffffff>;
    reusable;
    alignment = <0x0 0x400000>;
    size = <0x0 0xc00000>;
};
```

The following is the sample log for an aDSP CMA memory region reserved on boot:

```
[ 0.000000] OF: reserved mem: initialized node adsp-heap, compatible id
shared-dma-pool
[ 0.000000] OF: reserved mem: 0x00000000ff000000..0x00000000ffbfffff
(12288 KiB) map reusable adsp-heap
```

DMA-BUF heaps

DMA-BUF heaps are supported on the Qualcomm Linux distribution to allocate custom CMA heaps. With DMA-BUF heaps, a device file is present for each heap in the `/dev/dma_heap` file system. Apart from the system heap and the common CMA reserved heap, you can create your own CMA-type DMA-BUF heaps.

Supported heaps

The following table lists the heaps supported by default on the Qualcomm Linux distribution:

Heap name	Dev node	Description	Usage
System	<code>/dev/dma_heap/system</code>	The kernel creates the default DMA-BUF heap.	All generic use cases should follow the system heap that uses the common Linux memory management buddy allocator underneath.
Reserved	<code>/dev/dma_heap/reserved</code>	The default CMA-type heap created in the system, uses the default reserved CMA region.	If you need contiguous memory due to any constraints, use the CMA heap.
Custom CMA heaps	<code>/dev/dma_heap/my_cma_heap</code>	User-defined CMA-type heaps.	If you want to create your own CMA-type heap for specific use cases.

To use DMA-BUF heaps

Here is a sample program that demonstrates how to use the DMA-BUF system heap created by the Linux kernel in `/dev/dma_heap/system`:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <linux/dma-buf.h>

#define DMA_HEAP_NAME "system" int main()
{
    int fd, dma_buf_fd;
    struct dma_heap_export_info exp_info = {
        .heap_name = DMA_HEAP_NAME,
        .flags = 0,
    };
    struct dma_buf_sync sync_start = {
        .flags = DMA_BUF_SYNC_START,
    };
    struct dma_buf_sync sync_end = {
        .flags = DMA_BUF_SYNC_END,
    };

    fd = open("/dev/dma_heap/system", O_RDWR); if (fd < 0) {
        perror("open"); return -errno;
    }

    dma_buf_fd = ioctl(fd, DMA_HEAP_IOCTL_ALLOC, &exp_info); if (dma_buf_fd < 0) {
        perror("ioctl"); return -errno;
    }
    printf("Allocated DMA buffer with fd %d\n", dma_buf_fd); if
    (ioctl(dma_buf_fd, DMA_BUF_IOCTL_SYNC, &sync_start)) {
        perror("ioctl DMA_BUF_IOCTL_SYNC start"); return -errno;
    }

    // Do something with the buffer here

    if (ioctl(dma_buf_fd, DMA_BUF_IOCTL_SYNC, &sync_end)) { perror("ioctl
    DMA_BUF_IOCTL_SYNC end");
        return -errno;}
    re
    if (close(dma_buf_fd)) { perror("close"); return -errno;
    }

    if (close(fd)) {
```

```

perror("close"); return -errno;
}

return 0;
}

```

3.5 Scheduler

The scheduler decides the order in which the processes must run. The scheduler runs the processes available in the runqueue of each CPU.

The kernel baseline supports the standard Linux scheduler solution. The kernel uses [Energy Aware Scheduling \(EAS\)](#) to choose the right CPU for task placement based on the CPU energy consumption.

EAS overrides the CFS task wake-up balancing code. It uses the energy model (EM) of the platform and the per entity load tracking (PELT) signals to choose an energy-efficient target CPU during wake-up balance in a system with asymmetric CPU topology.

For basic scheduler documentation, see [Scheduler](#).

3.5.1 CPU topology and EAS

The Qualcomm SoC platforms have a heterogeneous CPU topology that is differentiated in terms of CPU capacity metrics used in EAS.

EAS uses the concept of 'capacity' to differentiate CPUs with different computing capabilities. The capacity of a CPU represents the amount of work that it finishes when running at its highest frequency compared to the most capable CPU of the system. Capacity values are normalized in a 1024 range (the most powerful CPU/cluster is configured at 1024).

EAS builds the capacity of a cluster based on the dhrystone million instructions per second (DMIPS) value specified in the CPU node and the maximum frequency supported by the cluster.

The following are the `sysfs` nodes for CPU topology and capacity:

Table 3-1 CPU topology and capacity

Commands	Purpose
<code>cat /sys/devices/system/cpu/cpu*/cpu_capacity</code>	To get the capacity associated with each CPU in the system.
<code>cat /sys/devices/system/cpu/cpufreq/policy*/related_cpus</code>	To get a list of the CPUs associated with each cluster in the system.

For more information on how a scheduler uses CPU capacity, see [Capacity Aware Scheduling](#).

3.5.2 Per entity load tracking

Per entity load tracking (PELT) is the load tracking mechanism to track load accounting for `sched_entities/groups/runqueues`.

PELT clock multiplier can be tuned to meet power and performance requirements.

The command line parameter `sched_pelt_multiplier` allows you to set a clock multiplier. The clock multiplier tunes the PELT ramp up/down speed to tune the system for the power and performance.

With lower half-life time, the CPU utilization of tasks is accumulated faster, and the tasks are placed on performance CPUs, leading to better performance with high power consumption. The CPU utilization also gets accumulated to select the higher CPU frequency within less time.

NOTE The half-life also affects how the task utilization ramps down. UTIL EST feature provides faster ramp down.

For more information on PELT, see [Per-entity load tracking](#).

3.5.3 SchedUtil governor

The SchedUtil CPU frequency governor is the default governor in Qualcomm platforms. SchedUtil predicts optimal operating points (OPPs) based on the CPU utilization, maintaining coherence between frequency requests and energy predictions.

The SchedUtil CPU frequency governor is tied to EAS and it tries to predict at which OPP all the CPUs run in the near future to estimate their energy consumption.

For more information on SchedUtil, see [Schedutil](#).

3.5.4 Utilization clamping (UCLAMP)

The UCLAMP allows performance management of tasks/task groups from the user space.

UCLAMP allows task or task group placement and performance hints or constraints. This mechanism is used to influence the scheduler placement decisions or to influence the frequency guidance of the cluster.

For more information on UCLAMP, see [UCLAMP](#).

3.6 Dynamic voltage and frequency scaling (DVFS)

Qualcomm kernel baseline supports several DVFS implementations to manage frequency scaling in a dynamic way corresponding to the system requirement.

3.6.1 CPU DVFS

You can select a governor to tune the system for power or performance while changing the CPUfreq governor (powersave/performance/schedutil). By default, CPU frequency governor is set to performance on Qualcomm Linux platforms.

Table 3-2 CPU DVFS variables

Variable	Description	Path
scaling_governor	Set the governor to performance.	echo performance > /sys/devices/system/cpu/cpufreq/policy*/scaling_governor
scaling_max_freq	Set the maximum frequency of the CPU cluster.	/sys/devices/system/cpu/cpufreq/policy*/scaling_min_freq or scaling_max_freq tunable
scaling_min_freq	Set the minimum frequency of the CPU cluster.	/sys/devices/system/cpu/cpufreq/policy*/scaling_min_freq or scaling_min_freq tunable

For more information on the CPUfreq, see <https://www.kernel.org/doc/Documentation/cpu-freq/governors.txt>.

3.6.2 Cache and memory DVFS

The static map DVFS governors aligns the CPU operating frequencies with cache or memory frequencies for level 3 cache (L3), last level cache controller (LLCC), and DDR RAM.

If CPU frequency is at maximum, either by setting the CPU frequency governor to performance, or due to load in the system, then the L3, LLCC, and DDR also run at maximum frequency.

The driver for this governor is present in the `drivers/cpufreq/qcom-cpufreq-hw.c` file, and static mapping is present in the `arch/arm64/boot/dts/qcom/sc7280.dtsi` file.

The other bandwidth monitor (BWMON) governor is used to vote for LLCC and DDR frequencies, based on measured traffic between CPU to LLCC and CPU to DDR. The driver for this governor is in the `drivers/soc/qcom/icc-bwmon.c` file.

3.7 Virtualization

Virtualization uses software to create an abstraction layer over computer hardware.

The abstraction layer allows the hardware elements of a single computer, such as processor, memory, and storage to be divided into multiple virtual computers, called as virtual machines (VMs). Each VM

runs its own operating system (OS) and behaves like an independent computer, though it is running on the same underlying computer hardware.

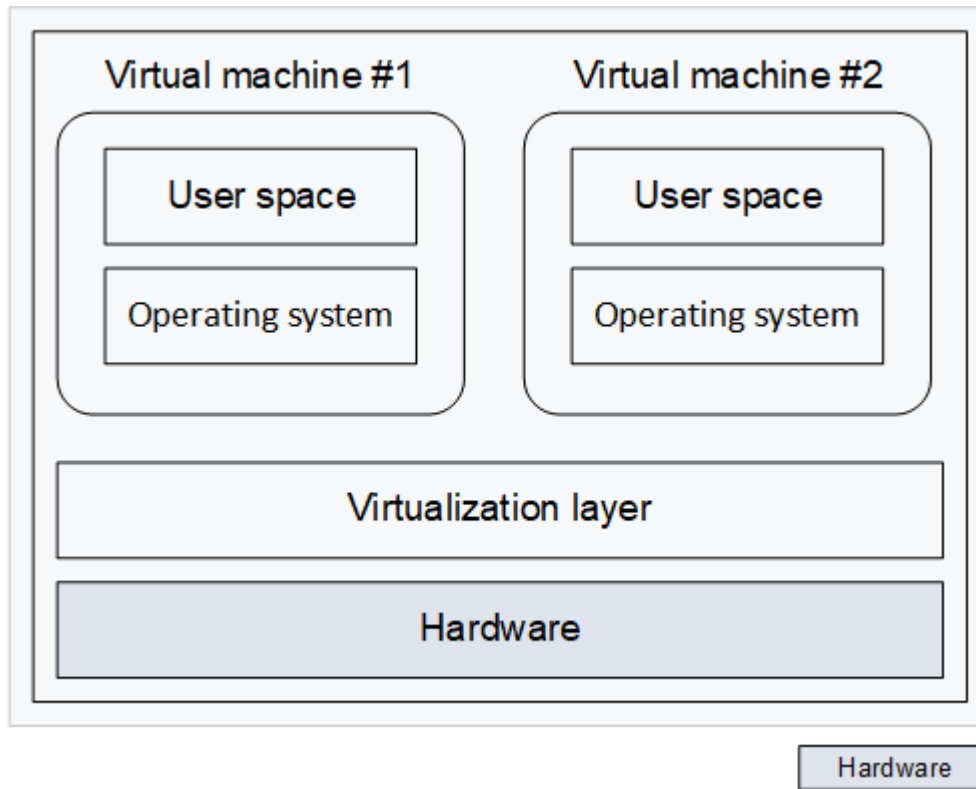


Figure 3-4 Virtualization

3.7.1 Qualcomm virtualization solution

Qualcomm uses Gnyah, a type-1 hypervisor designed for strong security, high performance, OS independence, and higher CPU privilege level.

Gnyah does not depend on any lower-privileged OS kernel or code for its core functionality to offer increased security. Gnyah is designed for isolated virtual machine use cases, and to support launching isolated virtual machines from a relatively less trusted host virtual machine.

The following image shows the Qualcomm virtualization solution architecture:

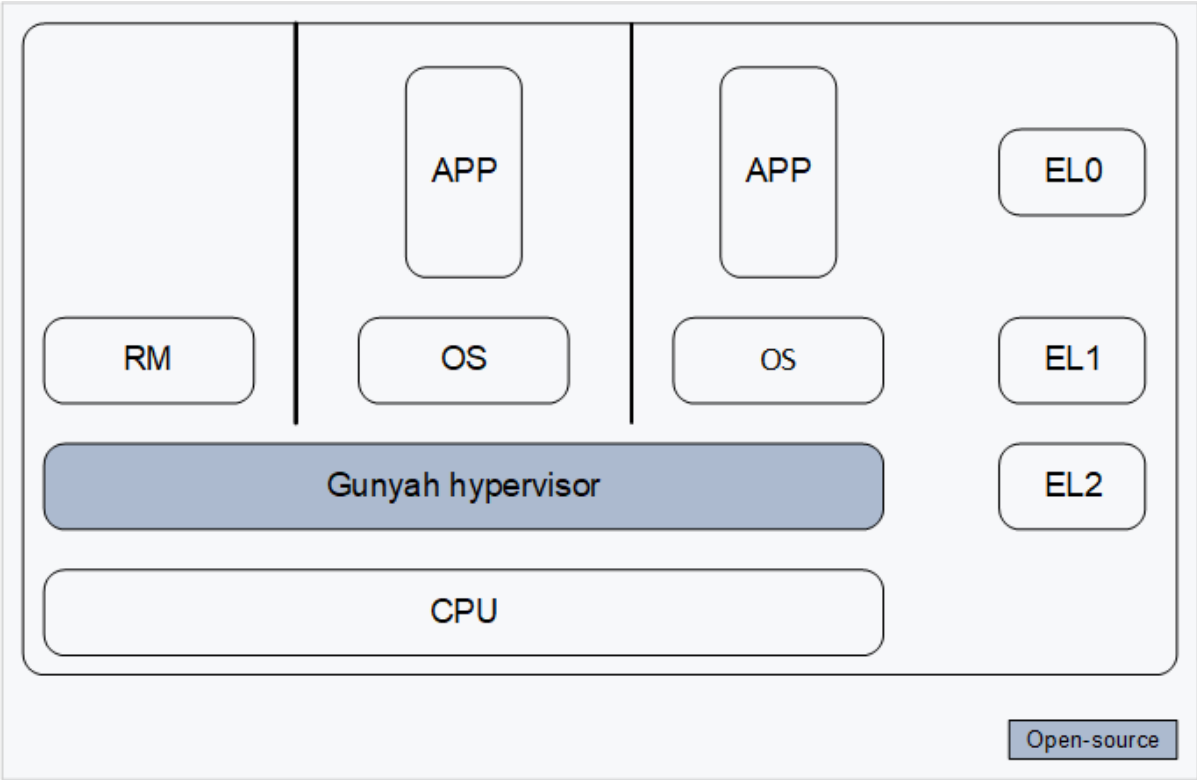


Figure 3-5 Qualcomm virtualization solution architecture

Gunyah provides the following features:

Table 3-3 Features of Gunyah

Features	Description
Threads and scheduling	Scheduler schedules virtual CPUs (VCPUs) on physical CPUs and enables time-sharing of the CPUs.
Memory management	Gunyah tracks memory ownership and memory usage under its control. Gunyah provides low-level dynamic memory management APIs on top of which the higher-level donation, lending, and sharing is built.
Interrupt virtualization	Gunyah manages interrupts that are routed directly to the assigned VM.
Inter-VM communication	There are several different mechanisms, such as message queue and doorbell, provided for communication between the VMs.
Device virtualization	Para-virtualization of devices is supported using inter-VM communication and virtio primitives. The hardware virtualization and emulation support low-level architecture features and devices, such as CPU timers and interrupt controllers.
Resource manager	Gunyah supports a root VM that initially owns all the VM memory and input/output resources. The Gunyah resource manager is the default-bundled root VM and provides high-level services including dynamic VM management, secure memory donation, lending, and sharing.

The Gunchah hypervisor support is added to the [crosvm](#) virtual memory monitor (VMM). The crosvm interacts with the Gunchah VM manager in the kernel through `/dev/gunchah` interface and creates a VM.

The following image shows the blocks in the Qualcomm virtualization solution:

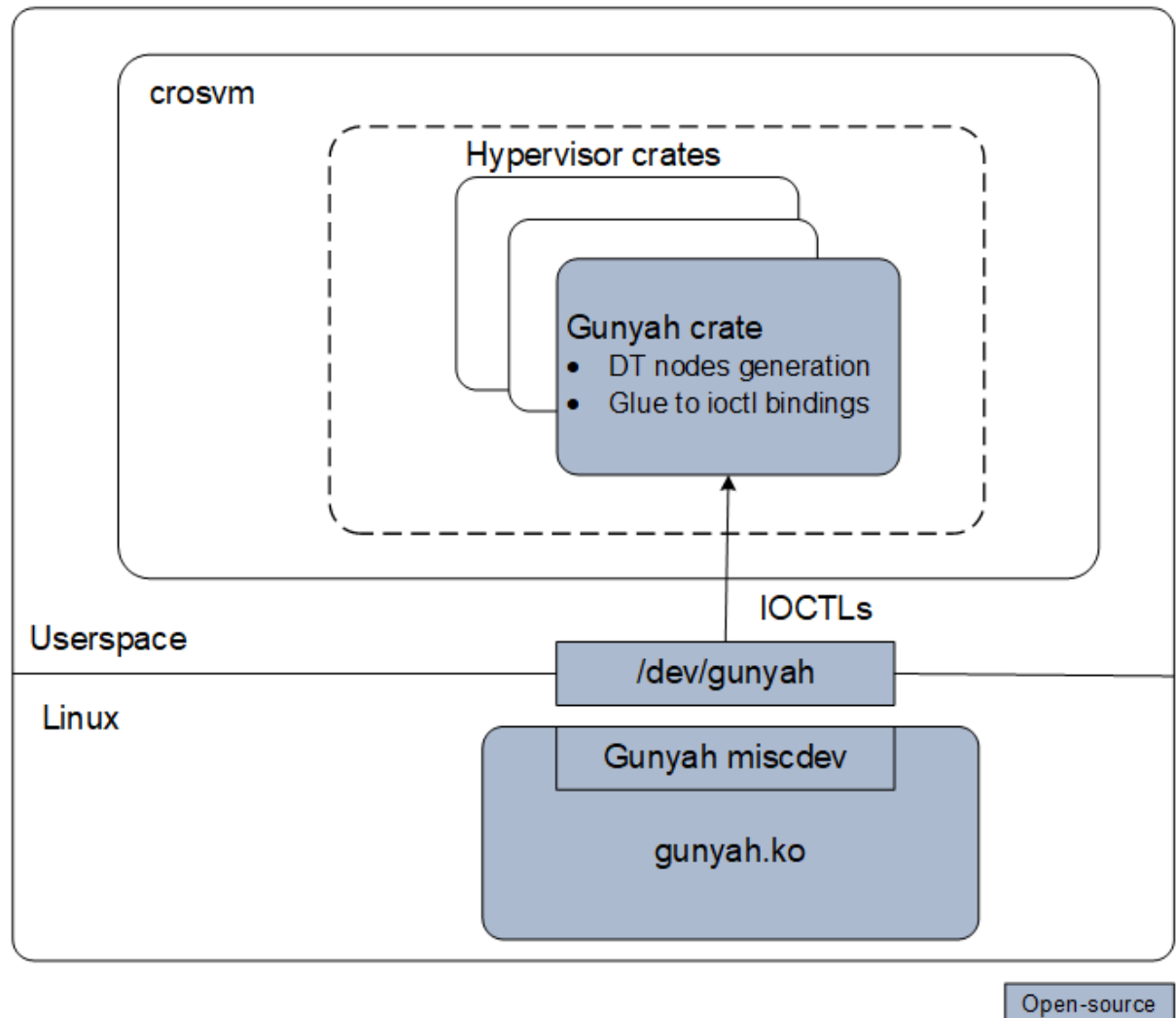


Figure 3-6 Qualcomm virtualization solution

For more information on type 1 hypervisor, see [gunyah-hypervisor](#).

3.7.2 Guest VM build support

Use the following procedure to compile the guest VM kernel and initrd images.

During the guest VM kernel compilation, both the guest kernel and the minimal `rootfs` are built. The `qcom_vm_defconfig` serves as the base configuration for compiling the guest kernel. The `qcom_vm_debug.config` build debug variants of the guest VM.

The `crosvm` serves as the VMM and interacts with the Gunchah VM manager in the kernel through the `/dev/gunchah` interface to create the virtual machines (VMs).

Build a guest VM

For more information on `crosvm` support, see [Secondary virtual machine](#).

After the build process completes, certain images are generated. These images get stored in the `<workspace>/build-qcom-wayland/tmp-glibc/deploy/images/qcm6490/qcom-console-image` directory.

Use the following command to compile the guest VM:

```
$ export SHELL=/bin/bash && MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment $ bitbake qcom-console-image
```

Guest VM initrd image

- The `svm-initramfs-qcom-image-qcm6490.cpio.gz` file is created for the guest VM. This image contains an initrd image, used during the boot process.
- The `svm-initramfs-qcom-image-qcm6490.cpio.gz` file is generated in the `build-qcom-wayland/tmp-glibc/deploy/images/qcm6490/svm-initramfs-qcom-image` directory.
- On the booted device, the image is stored in the `/var/gunyah/initrd.img` file within the system rootfs.

Guest VM kernel image

- `svm-initramfs-qcom-image-qcm6490.cpio.gz` image is generated for the secondary virtual machine (SVM), which is the Guest kernel image.
- The SVM kernel image is generated in the `build-qcom-wayland/tmp-glibc/deploy/images/qcm6490/linux-svm-kernel-qcom` directory.
- On the booted device, the image is in a `/var/gunyah/Image` file within the system rootfs.

Both the `initrd.img` and the SVM kernel images are packed in the `system.img` file. The final `system.img` file is found in the same directory as the other images `<workspace>/build-qcom-wayland/tmp-glibc/deploy/images/qcm6490/qcom-console-image`.

3.7.3 Launch guest virtual machine

Use `crosvm` VMM and the Gunyah hypervisor to launch the guest virtual machines.

Use the following command to launch the VM:

```
# /usr/bin/crosvm --log-level=debug --no-syslog run --disable-sandbox --
hypervisor \
                                                    gunyah --protected-vm-without-
firmware \
                                                    --
serial=type=stdout,hardware=virtio-console,console,stdin,num=1 \
                                                    --
serial=type=stdout,hardware=serial,earlycon,num=1 \
                                                    --initrd /var/gunyah/initrd.img --
no-balloon --no-rng \
                                                    --params "rw root=/dev/ram
earlyprintk=serial panic=0" \
                                                    /var/gunyah/Image
```

The following are the supported parameters for the `crosvm` command to launch guest VMs.

Table 3-4 Supported parameters

Parameters	Description
<code>/usr/bin/crosvm</code>	Shows the path of the <code>crosvm</code> binary.
<code>--log-level=debug</code>	Sets the logging level to debug while running <code>crosvm</code> .
<code>--no-syslog</code>	Disables syslog.
<code>--disable-sandbox</code>	Disables sandboxing.
<code>--hypervisor gunyah</code>	Specifies the hypervisor to be used (in this case, <code>gunyah</code>).
<code>--protected-vm-without-firmware</code>	Indicates that the VM is protected and does not require firmware.
<code>--serial=...</code>	Configures serial devices for input/output.
<code>--initrd /var/gunyah/initrd.img</code>	Specifies the initial RAM disk (initrd) image.
<code>--no-balloon</code>	Disables memory ballooning.
<code>--no-rng</code>	Disables the entropy source used to seed the guest OS entropy pool.
<code>--params "..."</code>	Specifies the kernel command line options.
<code>/var/gunyah/Image</code>	Specifies the path to the kernel image.

Run VM and view boot logs

The following code snippet shows the VM logs:

```
bash-5.1# cat /proc/cpuinfo
processor          : 0
BogoMIPS         : 38.40
Features          : fp asimd evtstrm aes pmull sha1 crc32 atomics cpuid
asimdrdm dcpop asimddp
CPU implementer   : 0x00
CPU architecture: 8
CPU variant       : 0x0
CPU part          : 0x048
CPU revision      : 0

bash-5.1# cat /proc/meminfo
MemTotal:         161656 kB
MemFree:          124940 kB
```

```

MemAvailable:      124032 kB
Buffers:           0 kB
Cached:            19816 kB
SwapCached:        0 kB
Active:            12 kB

```

The following code snippet shows the boot logs:

```

[2022-04-28T17:43:03.290477745+00:00 DEBUG crosvm::crosvm::sys::linux]
creating hypervisor: Gunyah { device: None }
[ 0.000000] Booting Linux on physical CPU 0x0000000000 [0x000f0480]
[ 0.000000] Linux version 6.6.0 (oe-user@oe-host) (aarch64-qcom-linux-gcc
(GCC) 11.4.0, GNU ld (GNU Binutils) 2.38.20220708) #1 SMP PREEMPT Wed Feb 7
04:56:59 UTC 2024
[ 0.000000] KASLR enabled
[ 0.000000] random: crng init done
[ 0.000000] Machine model: linux,dummy-virt
[ 0.000000] earlycon: uart8250 at MMIO 0x000000000000003f8 (options '')
[ 0.000000] printk: bootconsole [uart8250] enabled
[ 0.000000] efi: UEFI not found.
[ 0.000000] software IO TLB: Reserved memory: created restricted DMA pool
at 0x0000000090000000, size 64 MiB
[ 0.000000] OF: reserved mem: initialized node
restricted_dma_reserved@90000000, compatible id restricted-dma-pool
[ 0.000000] OF: reserved mem: 0x0000000090000000..0x0000000093ffffff
(65536 KiB) map non-reusable restricted_dma_reserved@90000000
[ 0.000000] NUMA: No NUMA configuration found
[ 0.000000] NUMA: Faking a node at [mem
0x0000000080000000-0x0000000093ffffff]
[ 0.000000] NUMA: NODE_DATA [mem 0x8ff509c0-0x8ff52fff]
[ 0.000000] Zone ranges:
[ 0.000000]   DMA      [mem 0x0000000080000000-0x0000000093ffffff]
[ 0.000000]   DMA32    empty
[ 0.000000]   Normal   empty
[ 0.000000] Movable zone start for each node
[ 0.000000] Early memory node ranges
[ 0.000000]   node    0: [mem 0x0000000080000000-0x0000000093ffffff]
[ 0.000000] Initmem setup node 0 [mem
0x0000000080000000-0x0000000093ffffff]
[ 0.000000] On node 0, zone DMA: 16384 pages in unavailable ranges
[ 0.000000] cma: Reserved 32 MiB at 0x000000008d600000 on node -1
[ 0.000000] psci: probing for conduit method from DT.
[ 0.000000] psci: PSCIv1.1 detected in firmware.
[ 0.000000] psci: Using standard PSCI v0.2 function IDs
[ 0.000000] psci: MIGRATE_INFO_TYPE not supported.
[ 0.000000] psci: SMC Calling Convention v1.3
[ 0.000000] percpu: Embedded 31 pages/cpu s86568 r8192 d32216 u126976
[ 0.000000] Detected PIPT I-cache on CPU0
[ 0.000000] CPU features: detected: GIC system register CPU interface

```

```
[ 0.000000] CPU features: detected: Spectre-v4
[ 0.000000] CPU features: kernel page table isolation forced ON by KASLR
[ 0.000000] CPU features: detected: Kernel page table isolation (KPTI)
[ 0.000000] alternatives: applying boot alternatives
[ 0.000000] Kernel command line: panic=-1 console=hvc0
earlycon=uart8250,mmio,0x3f8 rw root=/dev/ram earlyprintk=serial panic=0
[ 0.000000] Unknown kernel command line parameters "earlyprintk=serial",
will be passed to user space.
[ 0.000000] Dentry cache hash table entries: 65536 (order: 7, 524288
bytes, linear)
[ 0.000000] Inode-cache hash table entries: 32768 (order: 6, 262144 bytes,
linear)
[ 0.000000] Fallback order for Node 0: 0
[ 0.000000] Built 1 zonelists, mobility grouping on. Total pages: 80640
[ 0.000000] Policy zone: DMA
[ 0.000000] mem auto-init: stack:off, heap alloc:off, heap free:off
[ 0.000000] software IO TLB: area num 1.
[ 0.000000] software IO TLB: mapped [mem
0x0000000089600000-0x000000008d600000] (64MB)
[ 0.000000] Memory: 116344K/327680K available (14528K kernel code, 3938K
rwd data, 7112K rodata, 6080K init, 571K bss, 178568K reserved, 32768K cma-
reserved)
[ 0.000000] SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=1, Nodes=1
[ 0.000000] trace event string verifier disabled
[ 0.000000] rcu: Preemptible hierarchical RCU implementation.
[ 0.000000] rcu: RCU event tracing is enabled.
[ 0.000000] rcu: RCU restricting CPUs from NR_CPUS=256 to nr_cpu_ids=1.
[ 0.000000] Trampoline variant of Tasks RCU enabled.
[ 0.000000] Tracing variant of Tasks RCU enabled.
[ 0.000000] rcu: RCU calculated value of scheduler-enlistment delay is 25
jiffies.
[ 0.000000] rcu: Adjusting geometry for rcu_fanout_leaf=16, nr_cpu_ids=1
[ 0.000000] NR_IRQS: 64, nr_irqs: 64, preallocated irqs: 0
[ 0.000000] GICv3: 988 SPIs implemented
[ 0.000000] GICv3: 0 Extended SPIs implemented
[ 0.000000] Root IRQ handler: gic_handle_irq
[ 0.000000] GICv3: GICv3 features: 16 PPIs
[ 0.000000] GICv3: CPU0: found redistributor 0 region 0:0x000000003ffd0000
[ 0.000000] rcu: srcu_init: Setting srcu_struct sizes based on contention.
[ 0.000000] arch_timer: cp15 timer(s) running at 19.20MHz (virt).
[ 0.000000] clocksource: arch_sys_counter: mask: 0xffffffffffffffff
max_cycles: 0x46d987e47, max_idle_ns: 440795202767 ns
[ 0.000000] sched_clock: 56 bits at 19MHz, resolution 52ns, wraps every
4398046511078ns
[ 0.002616] arm-pv: using stolen time PV
[ 0.003945] Console: colour dummy device 80x25
[ 0.005300] Calibrating delay loop (skipped), value calculated using timer
```

```
frequency.. 38.40 BogoMIPS (lpj=76800)
[ 0.008475] pid_max: default: 32768 minimum: 301
[ 0.009923] LSM: initializing lsm=capability,selinux,integrity
[ 0.011702] SELinux: Initializing.
[ 0.012884] Mount-cache hash table entries: 1024 (order: 1, 8192 bytes,
linear)
[ 0.015180] Mountpoint-cache hash table entries: 1024 (order: 1, 8192
bytes, linear)
[ 0.017809] cacheinfo: Unable to detect cache hierarchy for CPU 0
[ 0.019820] RCU Tasks: Setting shift to 0 and lim to 1
rcu_task_cb_adjust=1.
[ 0.022107] RCU Tasks Trace: Setting shift to 0 and lim to 1
rcu_task_cb_adjust=1.
[ 0.024472] rcu: Hierarchical SRCU implementation.
[ 0.025891] rcu: Max phase no-delay instances is 1000.
[ 0.027767] EFI services will not be available.
[ 0.029190] smp: Bringing up secondary CPUs ...
[ 0.030713] smp: Brought up 1 node, 1 CPU
[ 0.032002] SMP: Total of 1 processors activated.
[ 0.033471] CPU features: detected: 32-bit ELO Support
[ 0.035048] CPU features: detected: Data cache clean to the PoU not
required for I/D coherence
[ 0.037742] CPU features: detected: Common not Private translations
[ 0.039678] CPU features: detected: CRC32 instructions
[ 0.041275] CPU features: detected: LSE atomic instructions
[ 0.043105] CPU features: detected: Privileged Access Never
[ 0.044891] CPU: All CPU(s) started at EL1
[ 0.046161] alternatives: applying system-wide alternatives
[ 0.050071] devtmpfs: initialized
[ 0.051500] clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff,
max_idle_ns: 7645041785100000 ns
[ 0.054767] futex hash table entries: 256 (order: 2, 16384 bytes, linear)
[ 0.057122] pinctrl core: initialized pinctrl subsystem
[ 0.058872] DMI not present or invalid.
[ 0.060334] NET: Registered PF_NETLINK/PF_ROUTE protocol family
[ 0.062534] DMA: preallocated 128 KiB GFP_KERNEL pool for atomic
allocations
[ 0.064964] DMA: preallocated 128 KiB GFP_KERNEL|GFP_DMA pool for atomic
allocations
[ 0.067353] DMA: preallocated 128 KiB GFP_KERNEL|GFP_DMA32 pool for atomic
allocations
[ 0.069943] audit: initializing netlink subsys (disabled)
[ 0.071632] audit: type=2000 audit(0.056:1): state=initialized
audit_enabled=0 res=1
[ 0.074141] thermal_sys: Registered thermal governor 'step_wise'
[ 0.074143] thermal_sys: Registered thermal governor 'power_allocator'
[ 0.076118] cpuidle: using governor menu
```

```
[ 0.079536] hw-breakpoint: found 6 breakpoint and 4 watchpoint registers.
[ 0.081693] ASID allocator initialised with 32768 entries
[ 0.083662] Serial: AMBA PL011 UART driver
[ 0.085582] Modules: 2G module region forced by
RANDOMIZE_MODULE_REGION_FULL
[ 0.087812] Modules: 0 pages in range for non-PLT usage
[ 0.087813] Modules: 516176 pages in range for PLT usage
[ 0.091914] HugeTLB: registered 1.00 GiB page size, pre-allocated 0 pages
[ 0.099837] HugeTLB: 0 KiB vmemmap can be freed for a 1.00 GiB page
[ 0.101815] HugeTLB: registered 32.0 MiB page size, pre-allocated 0 pages
[ 0.107895] HugeTLB: 0 KiB vmemmap can be freed for a 32.0 MiB page
[ 0.109875] HugeTLB: registered 2.00 MiB page size, pre-allocated 0 pages
[ 0.116064] HugeTLB: 0 KiB vmemmap can be freed for a 2.00 MiB page
[ 0.118084] HugeTLB: registered 64.0 KiB page size, pre-allocated 0 pages
[ 0.124190] HugeTLB: 0 KiB vmemmap can be freed for a 64.0 KiB page
[ 0.128423] ACPI: Interpreter disabled.
[ 0.132206] iommu: Default domain type: Translated
[ 0.133734] iommu: DMA domain TLB invalidation policy: strict mode
[ 0.135749] SCSI subsystem initialized
[ 0.140957] usbcore: registered new interface driver usbfs
[ 0.142744] usbcore: registered new interface driver hub
[ 0.144353] usbcore: registered new device driver usb
[ 0.149976] pps_core: LinuxPPS API ver. 1 registered
[ 0.151488] pps_core: Software ver. 5.3.6 - Copyright 2005-2007 Rodolfo
Giometti <giometti@linux.it>
[ 0.162194] PTP clock support registered
[ 0.163436] EDAC MC: Ver: 3.0.0
[ 0.164532] scmi_core: SCMI protocol bus registered
[ 0.166176] FPGA manager framework
[ 0.171350] Advanced Linux Sound Architecture Driver Initialized.
[ 0.173213] vgaarb: loaded
[ 0.174176] clocksource: Switched to clocksource arch_sys_counter
[ 0.235827] VFS: Disk quotas dquot_6.6.0
[ 0.237108] VFS: Dquot-cache hash table entries: 512 (order 0, 4096 bytes)
[ 0.239428] pnp: PnP ACPI: disabled
[ 0.241483] NET: Registered PF_INET protocol family
[ 0.243293] IP idents hash table entries: 8192 (order: 4, 65536 bytes,
linear)
[ 0.245817] tcp_listen_portaddr_hash hash table entries: 256 (order: 0,
4096 bytes, linear)
[ 0.248604] Table-perturb hash table entries: 65536 (order: 6, 262144
bytes, linear)
[ 0.251034] TCP established hash table entries: 4096 (order: 3, 32768
bytes, linear)
[ 0.253323] TCP bind hash table entries: 4096 (order: 5, 131072 bytes,
linear)
[ 0.255664] TCP: Hash tables configured (established 4096 bind 4096)
```

```
[ 0.257584] UDP hash table entries: 256 (order: 1, 8192 bytes, linear)
[ 0.259650] UDP-Lite hash table entries: 256 (order: 1, 8192 bytes, linear)
[ 0.261768] NET: Registered PF_UNIX/PF_LOCAL protocol family
[ 0.263829] RPC: Registered named UNIX socket transport module.
[ 0.265673] RPC: Registered udp transport module.
[ 0.267186] RPC: Registered tcp transport module.
[ 0.268641] RPC: Registered tcp-with-tls transport module.
[ 0.270380] RPC: Registered tcp NFSv4.1 backchannel transport module.
[ 0.272421] PCI: CLS 0 bytes, default 64
[ 0.273680] Unpacking initramfs...
[ 0.278241] kvm [1]: HYP mode not available
[ 0.279783] Initialise system trusted keyrings
[ 0.286209] workingset: timestamp_bits=42 max_order=16 bucket_order=0
[ 0.288474] squashfs: version 4.0 (2009/01/31) Phillip Lougher
[ 0.294313] NFS: Registering the id_resolver key type
[ 0.295881] Key type id_resolver registered
[ 0.297197] Key type id_legacy registered
[ 0.302225] nfs4filelayout_init: NFSv4 File Layout Driver Registering...
[ 0.304407] nfs4flexfilelayout_init: NFSv4 Flexfile Layout Driver
Registering...
[ 0.310289] 9p: Installing v9fs 9p2000 file system support
[ 0.320607] NET: Registered PF_ALG protocol family
[ 0.322022] Key type asymmetric registered
[ 0.330205] Asymmetric key parser 'x509' registered
[ 0.331833] Block layer SCSI generic (bsg) driver version 0.4 loaded
(major 245)
[ 0.334140] io scheduler mq-deadline registered
[ 0.342201] io scheduler kyber registered
[ 0.343459] io scheduler bfq registered
[ 0.353648] pci-host-generic 10000.pci: assigned reserved memory node
restricted_dma_reserved@90000000
[ 0.362220] pci-host-generic 10000.pci: host bridge /pci ranges:
[ 0.364055] pci-host-generic 10000.pci:      MEM
0x0002000000..0x0003ffffff -> 0x0002000000
[ 0.370193] pci-host-generic 10000.pci:      MEM
0x0094800000..0xfffffffffff -> 0x0094800000
[ 0.372927] pci-host-generic 10000.pci: Memory resource size exceeds max
for 32 bits
[ 0.382221] PCI: OF: PROBE_ONLY enabled
[ 0.383535] pci-host-generic 10000.pci: ECAM at [mem
0x00010000-0x0100ffff] for [bus 00]
[ 0.386134] pci-host-generic 10000.pci: PCI host bridge to bus 0000:00
[ 0.394279] pci_bus 0000:00: root bus resource [bus 00]
[ 0.395912] pci_bus 0000:00: root bus resource [mem 0x02000000-0x03ffffff]
[ 0.397972] pci_bus 0000:00: root bus resource [mem
0x94800000-0xfffffffffff]
[ 0.406329] pci 0000:00:00.0: [8086:1237] type 00 class 0x060000
```

```
[ 0.409065] pci 0000:00:01.0: [1af4:1043] type 00 class 0x00ff00
[ 0.419597] pci 0000:00:01.0: reg 0x10: [mem 0x02000000-0x02007fff]
[ 0.425686] Freeing initrd memory: 6464K
[ 0.428105] pci 0000:00:01.0: PME# supported from D0 D3hot D3cold
[ 0.430575] pci 0000:00:02.0: [1b36:0011] type 00 class 0xffff00
[ 0.432661] pci 0000:00:02.0: reg 0x10: [mem 0x02008000-0x0200800f]
[ 0.436076] pci 0000:00:00.0: Limiting direct PCI/PCI transfers
[ 0.438471] virtio-pci 0000:00:01.0: assigned reserved memory node
restricted_dma_reserved@90000000
[ 0.443123] Serial: 8250/16550 driver, 4 ports, IRQ sharing enabled
[ 0.445534] 2e8.U6_16550A: ttyS0 at MMIO 0x2e8 (irq = 14, base_baud =
115200) is a 16550A
[ 0.448422] 2f8.U6_16550A: ttyS1 at MMIO 0x2f8 (irq = 14, base_baud =
115200) is a 16550A
[ 0.451159] 3e8.U6_16550A: ttyS2 at MMIO 0x3e8 (irq = 15, base_baud =
115200) is a 16550A
[ 0.453983] 3f8.U6_16550A: ttyS3 at MMIO 0x3f8 (irq = 15, base_baud =
115200) is a 16550A
[ 0.456935] msm_serial: driver initialized
[ 0.460508] printk: console [hvc0] enabled
[ 0.461772] printk: bootconsole [uart8250] disabled
[ 0.475305] loop: module loaded
[ 0.475957] megasas: 07.725.01.00-rc1
[ 0.476374] tun: Universal TUN/TAP device driver, 1.6
[ 0.477187] VFIO - User Level meta-driver version: 0.3
[ 0.477691] usbcore: registered new interface driver usb-storage
[ 0.478075] i2c_dev: i2c /dev entries driver
[ 0.478575] sdhci: Secure Digital Host Controller Interface driver
[ 0.479311] sdhci: Copyright(c) Pierre Ossman
[ 0.479612] sdhci-pltfm: SDHCI platform and OF driver helper
[ 0.479932] ledtrig-cpu: registered to indicate activity on CPUs
[ 0.480343] SMCCC: SOC_ID: ARCH_SOC_ID not implemented, skipping ....
[ 0.480864] usbcore: registered new interface driver usbhid
[ 0.481329] usbhid: USB HID core driver
[ 0.482042] NET: Registered PF_PACKET protocol family
[ 0.482342] 9pnet: Installing 9P2000 support
[ 0.482635] Key type dns_resolver registered
[ 0.484383] registered taskstats version 1
[ 0.484775] Loading compiled-in X.509 certificates
[ 0.486896] page_owner is disabled
[ 0.487629] Key type fscrypt registered
[ 0.487943] Key type fscrypt-provisioning registered
[ 0.488391] clk: Disabling unused clocks
[ 0.488759] ALSA device list:
[ 0.489050]   No soundcards found.
[ 0.490311] Freeing unused kernel memory: 6080K
[ 0.498264] Run /init as init process
```

```
Starting version 250.5+
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
bash-5.1#
```

Launch a VM with 2 CPUs and default RAM

Launch VM: To launch guest VM with two CPU, use the `--cpus num-cores=2` parameter:

```
#/usr/bin/crosvm --log-level=debug --no-syslog run --disable-sandbox --
hypervisor \
                                gunyah --protected-vm-without-
firmware \
                                --cpus num-cores=2 \
                                --
serial=type=stdout,hardware=virtio-console,console,stdin,num=1 \
                                --
serial=type=stdout,hardware=serial,earlycon,num=1 \
                                --initrd /var/gunyah/initrd.img --
no-balloon --no-rng \
                                --params "rw root=/dev/ram
earlyprintk=serial panic=0" \
                                /var/gunyah/Image
```

The following code snippet shows the VM logs:

```
bash-5.1# cat /proc/cpuinfo
processor       : 0
BogoMIPS      : 38.40
Features      : fp asimd evtstrm aes pmull sha1 crc32 atomics cpuid
asimdrdm dcpop asimddp
CPU implementer : 0x00
CPU architecture: 8
CPU variant    : 0x0
CPU part       : 0x048
CPU revision   : 0

processor       : 1
BogoMIPS      : 38.40
Features      : fp asimd evtstrm aes pmull sha1 crc32 atomics cpuid
asimdrdm dcpop asimddp
CPU implementer : 0x00
CPU architecture: 8
CPU variant    : 0x0
CPU part       : 0x048
CPU revision   : 0

bash-5.1# cat /proc/meminfo
MemTotal:      161656 kB
MemFree:       124940 kB
MemAvailable:  124032 kB
```

```

Buffers:          0 kB
Cached:          19816 kB
SwapCached:      0 kB
Active:          12 kB

```

Launch VM with 2 CPUs and 512MB RAM

Launch VM: To launch guest VM with one CPU and 512 MB RAM, use the `--cpus num-cores=1 --mem size=512` parameters.

```

#/usr/bin/crosvm --log-level=debug --no-syslog run --disable-sandbox --
hypervisor \
                                gunyah --protected-vm-without-
firmware \
                                --cpus num-cores=2 --mem
size=512 \
                                --
serial=type=stdout,hardware=virtio-console,console,stdin,num=1 \
                                --
serial=type=stdout,hardware=serial,earlycon,num=1 \
                                --initrd /var/gunyah/initrd.img --
no-balloon --no-rng \
                                --params "rw root=/dev/ram
earlyprintk=serial panic=0" \
                                /var/gunyah/Image

```

The following code snippet shows the VM logs:

```

bash-5.1# cat /proc/cpuinfo
processor          : 0
BogoMIPS          : 38.40
Features           : fp asimd evtstrm aes pmull sha1 crc32 atomics cpuid
asimdrdm dcpop asimddp
CPU implementer   : 0x00
CPU architecture : 8
CPU variant       : 0x0
CPU part          : 0x048
CPU revision      : 0

processor          : 1
BogoMIPS          : 38.40
Features           : fp asimd evtstrm aes pmull sha1 crc32 atomics cpuid
asimdrdm dcpop asimddp
CPU implementer   : 0x00
CPU architecture : 8
CPU variant       : 0x0
CPU part          : 0x048
CPU revision      : 0

bash-5.1# cat /proc/meminfo
MemTotal:         418304 kB

```

```
MemFree:          378356 kB
MemAvailable:     376048 kB
Buffers:          0 kB
Cached:           20308 kB
SwapCached:       0 kB
Active:           0 kB
```

Launch VM with block device

Use the following command to launch the VM:

```
#/usr/bin/crosvm --log-level=debug --no-syslog run --disable-sandbox --
hypervisor \
                                gunyah --protected-vm-without-
firmware \
                                --block disk.img \
                                --cpus num-cores=2 --mem
size=256 \
                                --
serial=type=stdout,hardware=virtio-console,console,stdin,num=1 \
                                --
serial=type=stdout,hardware=serial,earlycon,num=1 \
                                --initrd /var/gunyah/initrd.img --
no-balloon --no-rng \
                                --params "rw root=/dev/ram
earlyprintk=serial panic=0" \
                                /var/gunyah/Image
```

- Use `--block` flag to specify the block device name (in this case, `disk.img`).
- This creates `/dev/vda`, `/dev/vdb`, and so on, in the guest VM.

```
bash-5.1# ls -al /dev/block/
total 0
drwxr-xr-x  2 root  root      220 Jan  1 00:00 .
drwxr-xr-x  9 root  root     2980 Jan  1 00:00 ..
lrwxrwxrwx  1 root  root         6 Jan  1 00:00 254:0 -> ../vda
lrwxrwxrwx  1 root  root         8 Jan  1 00:00 7:0 -> ../loop0
lrwxrwxrwx  1 root  root         8 Jan  1 00:00 7:1 -> ../loop1
```

This device can be mounted in the guest VM with the mount command, `mount -t ext4 /dev/vda /mount-point`.

```
bash-5.1# mkdir /data
bash-5.1# mount -t ext4 /dev/vda /data
[ 112.311376] EXT4-fs (vda): mounted filesystem
4b577765-65ad-4fd8-910c-0eac437d8afb r/w with ordered data mode. Quota mode:
none.
```

3.8 Real-time (RT) kernel overview

A real-time system is a deterministic system, where response to an event is expected in a set time.

A system is classified as compatible with RT if:

- It is devoid of unbounded latency.
- The maximum response time is calculated with precision.
- It meets the set criteria for scheduling of tasks (latency and deadline).

Linux can be considered as a real-time operating system (RTOS) in which real-time tasks have well-defined periodic execution cycles (cycle time), and meets execution criteria within specified limits (jitter).

When considering Linux as RTOS, apply `PREEMPT_RT` patches and enable configurations for a fully preemptible kernel.

The RT kernel maintainers maintain out-of-tree `PREEMPT_RT` patches for every kernel version released.

To install the patches, see [Versions of PREEMPT_RT patches](#).

NOTE The real-time support is for kernel space process and not for user space.



Figure 3-7 Build sequence

Set up workspace

Qualcomm kernel supports the v6.6 LTS RT kernel, which is maintained through the Yocto recipe in the `meta-qcom-realtime` layer in the `recipes-kernel/linux/linux-kernel-qcom-rt_6.6.bb` file.

The Linux RT kernel recipe in Qualcomm Linux is referred as `linux-kernel-qcom-rt`.

For more information on cloning the workspace and get all meta layers to use Qualcomm RT Linux, see [How to sync and build with real-time Linux?](#)

Enable real-time kernel

Qualcomm Linux `meta-qcom-realtime` layer supports a `linux-kernel-qcom-rt` recipe that fetches and builds the Linux kernel for the supported machines by default.

The `meta-qcom-realtime` layer applies the changes on top of the existing layer. During the kernel build, `meta-qcom-hwe`, `meta-qcom-realtime` layer appends the upstream `PREEMPT_RT` patches based on the kernel version, and enables the real-time configurations.

```
require recipes-kernel/linux/linux-kernel-qcom_6.6.bb
```

```
SECTION = "RT kernel"
SUMMARY = "Linux Real time kernel for QCOM devices"
DESCRIPTION = "Recipe to build real time Linux kernel"

SRC_URI:append = "https://cdn.kernel.org/pub/linux/kernel/projects/rt/6.6/
older/patch-6.6.14-rt21.patch.gz;md5sum=91969a704a73aa918c89d3027bdd3634 \
file://qcom_rt.cfg \
file://0001-arch-Kconfig-Add-RT-kernel-support.patch \

S = "${WORKDIR}/kernel"
KERNEL_CONFIG_FRAGMENTS:append = " ${WORKDIR}/qcom_rt.cfg"
```

For more information on supported machines, see [Identify supported Qualcomm machines](#).

NOTE RT kernel is not supported on QCS5430.

Configure real-time Kernel

Use the following procedure to configure the RT kernel.

```
SRC_URI:append = "https://cdn.kernel.org/pub/linux/kernel/projects/rt/6.6/
older/patch-6.6.14-rt21.patch.gz;md5sum=91969a704a73aa918c89d3027bdd3634 \
file://qcom_rt.cfg \
file://0001-arch-Kconfig-Add-RT-kernel-support.patch \
```

- To apply the external configurations on the RT kernel:
 - Maintain the configuration file in the meta-qcom-realtime/recipes-kernel/linux/linux-kernel-qcom-rt recipe.
 - Append the configuration file to KERNEL_CONFIG_FRAGMENTS in meta-qcom-realtime/recipes-kernel/linux/linux-kernel-qcom_6.6.bb file.

- To add a configuration fragment to the RT kernel:

```
KERNEL_CONFIG_FRAGMENTS:append = " ${WORKDIR}/qcom_rt.cfg"
```

- To modify the kernel command line, add a kernel command line in the conf/layer.conf file.
- The following example shows how to modify a command line:

```
KERNEL_CMDLINE_EXTRA = "root=/dev/disk/by-partlabel/system rw rootwait
console=ttyMSM0,115200n8 pcie_pme=noms earlycon idle=poll skew_tick=1
rcu_nocbs=1-3 rcu_nocb_poll nohz_full=1-3 irqaffinity=4-7 isolcpus=1-3"
```

Kernel configurations

Optional and mandatory kernel configurations are used in the RT kernel.

Use CONFIG_PREEMPT_RT to enable full preemption in the RT kernel.

The CONFIG_PREEMPT_RT is enabled by default as part of the linux-kernel-qcom-rt_6.6.bb recipe.

The following example shows the kernel configuration:

```
# CONFIG_PREEMPT_RT
$ zcat proc/config.gz | grep CONFIG_PREEMPT
# CONFIG_PREEMPT_NONE is not set
# CONFIG_PREEMPT_VOLUNTARY is not set
# CONFIG_PREEMPT is not set
CONFIG_PREEMPT_RT=y
```

Use the following configurations to optimize the RT kernel response:

```
# CONFIG_NO_HZ
$ zcat proc/config.gz | grep NO_HZ
CONFIG_NO_HZ_COMMON=y
# CONFIG_NO_HZ_IDLE is not set
CONFIG_NO_HZ_FULL=y
# CONFIG_NO_HZ is not set
```

```
#CONFIG_CPUSETS
$ zcat proc/config.gz | grep CPUSETS
CONFIG_CPUSETS=y
```

Set the following kernel configuration options when affinizing the RT task:

- `CONFIG_NO_HZ_FULL` - When enabled, it configures the kernel to avoid sending scheduling-clock interrupts to CPUs with a single runnable task.
- `CONFIG_CPUSETS` - Use the `CONFIG_CPUSETS` configuration option to enable `cpuset`, where the CPU is grouped to form a set.

Use the `CONFIG_CPUSETS` configuration option when affinizing the RT task to a set of CPUs.



Figure 3-8 RT kernel verification

Build a real-time kernel

Use the following commands to compile the RT kernel:

```
$ MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment

#verify realtime layer is present

$ bitbake-layers show-layers | grep realtime
    meta-qcom-realtime    /layers/meta-qcom-realtime  12

# compile
$ bitbake qcom-console-image
```

Testing real-time kernel

The suite of tests are available in the Linux foundation RT test suite.

The cyclic test determines the best and worst case latencies for an RT application.

Access the RT test suite source code at [rt-tests/rt-tests.git](https://github.com/rt-tests/rt-tests.git).

To include the RT test suites as a part of the image, change the `layer.conf` file in the standard Yocto build:

```
IMAGE_INSTALL:append = "rt-tests numactl"
```

Run the following cyclic test:

```
$ cgexec -g cpuset:core1-3 cyclicttest -a 1-3 -t 3 -m -1 100000000 -i 1000 -p
99 -h 800
```

For more information on the RT test, see [RT-Tests](#).

Debug RT kernel

The following are the methods for troubleshooting issues in the RT kernel:

- How to verify that the underlying kernel is real-time?

After the boot is complete, run the following commands to verify the kernel type:

```
$ uname -r
6.6-rt15
$ uname -v
SMP PREEMPT_RT
```

- How to make drivers RT compliant?

Ensure that synchronization primitives do not break RT assumptions.

The following are examples of the scenarios that can break RT assumptions and cause unexpected system behavior:

```
/* Acquiring a preemptible lock in non preemptible context */

preempt_disable( )
.....
spin_lock( )

/* Acquiring a preemptible lock in non preemptible context /

raw_spin_lock( )
.....
spin_lock( )

/* Acquiring a non preemptible lock in preemptible context /

local_lock_irq( )
....
raw_spin_lock( )
```

- Debug RT kernel configurations

The following are the debug configurations for the RT kernel:

- `CONFIG_DEBUG_ATOMIC_SLEEP` - Checks for sleep inside an atomic section.
- `CONFIG_PROVE_RAW_LOCK_NESTING` - Enables the `raw_spinlock` vs. `spinlock` nesting checks to ensure that the lock nesting rules for `PREEMPT_RT` enabled kernels are not violated.

While debugging, you can switch from an RT kernel to a non-RT kernel. To switch, make changes to the `layer.conf` file to change virtual/kernel settings to the `linux-kernel-qcom` and compile again.

Run the following commands to make the changes:

```
- PREFERRED_PROVIDER_virtual/kernel = "linux-kernel-qcom-rt"
+ PREFERRED_PROVIDER_virtual/kernel = "linux-kernel-qcom"
```

For more information on locking primitives with the RT kernel, see [Lock types and their rules — The Linux Kernel documentation](#).

Tuning RT kernel

Tune the RT kernel to achieve a deterministic latency to RT tasks in the device.

The CPU cores that run RT tasks must be set to run at maximum operating frequency while preventing thermal mitigation of CPU frequency. For example, in an idle sleep scenario, RT tasks face scheduling latency due to CPU wake time delay.

Use the following system configuration for a system with eight CPU cores.

- RT tasks as outlined in the following example are affined to cores 1-3.
- CPU cores 1-3 run in isolated mode with all read-copy-update (RCU) affined to cores 4-7.
- IRQs are affined to cores 4-7 in the kernel command line arguments.
- System sleep is disabled for CPU cores 1-3 and is set to run at maximum frequency.

```
# Disable low power mode and sleep for CPU 1-3
$ echo 1 > /sys/devices/system/cpu/cpu1/cpuidle/state2/disable
$ echo 1 > /sys/devices/system/cpu/cpu1/cpuidle/state1/disable
$ echo 1 > /sys/devices/system/cpu/cpu1/cpuidle/state0/disable
$ echo 1 > /sys/devices/system/cpu/cpu2/cpuidle/state0/disable
$ echo 1 > /sys/devices/system/cpu/cpu2/cpuidle/state1/disable
$ echo 1 > /sys/devices/system/cpu/cpu2/cpuidle/state2/disable
$ echo 1 > /sys/devices/system/cpu/cpu3/cpuidle/state2/disable
$ echo 1 > /sys/devices/system/cpu/cpu3/cpuidle/state1/disable
$ echo 1 > /sys/devices/system/cpu/cpu3/cpuidle/state0/disable

# Enable max freq for cpu group 0-3
$ echo 1958400 > /sys/devices/system/cpu/cpufreq/policy0/scaling_min_freq

$ echo performance > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
```

```
# form cgroup for core 0-3
$ mkdir /sys/fs/cgroup/cpuset/core0/
$ mkdir /sys/fs/cgroup/cpuset/core1/
$ mkdir /sys/fs/cgroup/cpuset/core2/
$ mkdir /sys/fs/cgroup/cpuset/core3/

$ echo 0 > /sys/fs/cgroup/cpuset/core0/cpuset.mems
$ echo 0 > /sys/fs/cgroup/cpuset/core1/cpuset.mems
$ echo 0 > /sys/fs/cgroup/cpuset/core2/cpuset.mems
$ echo 0 > /sys/fs/cgroup/cpuset/core3/cpuset.mems

$ echo 0 > /sys/fs/cgroup/cpuset/core0/cpuset.cpus
$ echo 1 > /sys/fs/cgroup/cpuset/core1/cpuset.cpus
$ echo 2 > /sys/fs/cgroup/cpuset/core2/cpuset.cpus
$ echo 3 > /sys/fs/cgroup/cpuset/core3/cpuset.cpus

$ mkdir /sys/fs/cgroup/cpuset/core1-3/
$ echo 0 > /sys/fs/cgroup/cpuset/core1-3/cpuset.mems
$ echo 1-3 > /sys/fs/cgroup/cpuset/core1-3/cpuset.cpus
```

The following example shows how to add a kernel command line parameter to have no RCU callbacks in `layer.conf`:

- CPU cores 1-3
- IRQ affine to core 4-7
- Isolate CPU 1-3

```
KERNEL_CMDLINE_EXTRA = "root=/dev/disk/by-partlabel/system rw rootwait  
console=ttyMSM0,115200n8 pcie_pme=noms earlycon skew_tick=1 rcu_nocbs=1-3  
rcu_nocb_poll nohz_full=1-3 irqaffinity=4-7 isolcpus=1-3"
```

4 Yocto support

Qualcomm Linux `meta-qcom-hwe` layer supports `linux-kernel-qcom` recipe that fetches and builds the Linux kernel for supported machine.

For more information on available recipes, see [Build the Yocto image recipes and kernel configurations](#).

4.1 Kernel source

Kernel source can be fetched from the Codelinaro repository as a part of BitBake builds.

Yocto kernel

```
$ MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment
$ bitbake qcom-console-image

# kernel source is downloaded under following location
$ ls build-qcom-wayland/tmp-glibc/work-shared/qcm6490/kernel-source/
arch      Documentation  ipc          MAINTAINERS  samples      virt
block     drivers       Kbuild      Makefile     scripts
certs     fs            Kconfig     mm           security
COPYING   include       kernel      net          sound
CREDITS   init          lib         README       tools
crypto    io_uring      LICENSES    rust         usr
```

For more information on the setup and obtaining a source, see the [Qualcomm Linux Build Guide](#).

4.2 Kernel recipe

Qualcomm Linux maintains `meta-qcom-hwe` layer and hosts kernel recipe at `meta-qcom-hwe/recipes-kernel/linux/linux-kernel-qcom_6.6.bb` file.

The `PREFERRED_PROVIDER_virtual/kernel` is defined as `linux-kernel-qcom` in the `meta-qcom-hwe/conf/machine/include/qti-common.inc` file. All references to `virtual/kernel` point to `linux-kernel-qcom` recipe file.

For the kernel recipe, run the following commands:

```
$ MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment
$ bitbake qcom-console-image
```

```
# kernel recipe location
$ ls -l meta-qcom-hwe/recipes-kernel/linux/
linux-kernel-headers-install_6.6.bb
linux-kernel-qcom
linux-kernel-qcom_6.6.bb
linux-kernel-qcom-headers_6.6.bb
```

4.3 Unpack kernel source

The kernel source can be unpacked for development and customization after cloning the meta layer.

The following set of commands are used to unpack the source code:

```
# unpack kernel
$ devtool modify linux-kernel-qcom
# Fetch and set up kernel repository at build-qcom-wayland/workspace/sources/
linux-kernel-qcom
# Creates a bbappend recipe at build-qcom-wayland/workspace/append/linux-
kernel-qcom_6.6.bbappend
# and also adds build-qcom-wayland/workspace to conf/bblayers.conf
$ ls build-qcom-wayland/workspace/
append/      conf/        .devtool_md5  README        sources/

$ ls build-qcom-wayland/workspace/append
linux-kernel-qcom_6.6.bbappend

$ ls build-qcom-wayland/workspace/conf
layer.conf

# kernel source repository
$ ls build-qcom-wayland/workspace/sources/
linux-kernel-qcom
```

When the commands are run, BitBake picks the kernel from the workspace and applies the bbappend recipe.

4.4 Make kernel changes

Kernel changes or customizations are done in the build-qcom-wayland/workspace/sources/linux-kernel-qcom workspace.

To modify the kernel configuration, use the following commands:

```
# Modify sources in build-qcom-wayland/workspace/sources/linux-kernel-qcom,
or update kernel configuration by running makemenuconfig
$ devtool menuconfig linux-kernel-qcom

# config fragment is updated in following location
```

```
$ ls build-qcom-wayland/workspace/sources/linux-kernel-qcom/oe-local-files/  
devtool-fragment.cfg  
devtool-fragment.cfg
```

```
# you may do other changes to kernel and commit
```

4.5 Commit kernel changes

Use the following Git commands to commit your changes:

```
$ cd build-qcom-wayland/workspace/sources/linux-kernel-qcom  
$ git add .  
$ git commit -s -m "my changes"
```

4.6 Build kernel image

After the changes are done, use the following procedures to build the kernel and the image:

```
# to build kernel use  
$ devtool build linux-kernel-qcom  
  
# and following to build the image  
$ devtool build-image qcom-console-image  
  
# built images are produced in standard location  
$ ls build-qcom-wayland/tmp-glibc/deploy/images/qcm6490/
```

4.7 Maintain kernel changes

Use Devtool to develop and export the patch and create an append file in the `meta-mylayer` layer. To create your own layer, first add it to the host kernel `bbappend` files and make changes.

Run the following commands to create and append kernel patches in the meta layer:

```
# create your own layer first and add it to host kernel bbappend and changes  
$ bitbake-layers create-layer ~/meta-mylayer  
# mkdir -p ~/meta-mylayer/recipes-kernel/linux/linux-kernel-qcom  
$ bitbake-layers add-layer ~/meta-mylayer  
  
# following would update the meta-mylayer recipe and the change  
$ devtool finish linux-kernel-qcom ~/meta-mylayer  
  
# devtool finish shall populate the meta-mylayer recipes along with patches,  
  
$ ls -R meta-mylayer/recipes-kernel/  
meta-mylayer/recipes-kernel/:  
linux  
  
meta-mylayer/recipes-kernel/linux:
```

```
linux-kernel-qcom  linux-kernel-qcom_%.bbappend

meta-mylayer/recipes-kernel/linux/linux-kernel-qcom:
0001-my-patch.patch  devtool-fragment.cfg

$ less meta-mylayer/recipes-kernel/linux/linux-kernel-qcom_%.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"

SRC_URI += "file://devtool-fragment.cfg \
            file://0001-my-patch.patch"
```

After executing, `devtool finish` command, the `~/meta-mylayer` layer is updated with the following:

- Corresponding kernel changes hosted as patches in `meta-mylayer/recipes-kernel/linux/linux-kernel-qcom/*.patch`.
- Add and update `meta-mylayer/recipes-kernel/linux/linux-kernel-qcom_%.bbappend SRC_URI` with these patches.
- `build-qcom-wayland/workspace` can be removed after development is completed.

4.8 Clean up the workspace

Use the following Devtool and BitBake commands to clean up the workspace:

```
# to remove layer
$ bitbake-layers remove-layer ~/meta-mylayer

# to clean complete work space
$ bitbake -c cleanall recipe

# reset using Devtool
$ devtool reset linux-kernel-qcom

# manually delete the workspace directory
$ rm -rf build-qcom-wayland/workspace/sources/linux-kernel-qcom
```

5 Customize

The SoC and board device tree support is hosted in the kernel source at `arch/arm64/boot/dts/qcom`.

5.1 Kernel development

Use the following kernel recipes to customize the kernel and generate patches during kernel development:

```
# setup the workspace and get the sources following build guide
# setup the environment

$ MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment

# create your own layer to host changes
$ bitbake-layers create-layer ~/meta-mylayer
$ bitbake-layers add-layer ~/meta-mylayer

# use devtool to setup kernel source for development
$ devtool modify linux-kernel-qcom

# do the development , commit changes, build and test
$ devtool build linux-kernel-qcom
$ devtool build-image qcom-console-image

# built images are produced in standard location
$ ls build-qcom-wayland/tmp-glibc/deploy/images/qcm6490/

# generate the patches and update layer
$ devtool finish linux-kernel-qcom ~/meta-mylayer
```

For more information on customizing the Qualcomm Linux kernel recipe, see [Kernel recipe](#).

For information on Yocto provisions, host patches, and how to apply them, see [Yocto Project Linux Kernel Development](#).

5.2 Kernel configurations

Yocto build system is used for modifying the kernel configuration, while invoking `menuconfig`.

Run the following commands to modify the kernel configuration:

```
$ MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment
$ bitbake linux-kernel-qcom -c menuconfig

# Above would update .config in kernel build directory build-qcom-wayland/tmp-
glibc/work/qcm6490-qcom-linux/linux-kernel-qcom/6.6-r0/build/
# one can create a config fragment for modifications made by issuing following

$ bitbake linux-kernel-qcom -c diffconfig

# Above would create fragment.cfg in build directory build-qcom-wayland/tmp-
glibc/work/qcm6490-qcom-linux/linux-kernel-qcom/6.6-r0/
```

Alternatively, use the Devtool to modify the kernel configuration:

```
$ devtool modify linux-kernel-qcom
$ devtool menuconfig linux-kernel-qcom
$ devtool finish linux-kernel-qcom ~/meta-mylayer

# this would create a config fragment as a patch and update in your meta layer

For information on Yocto related details on kernel configuration, see Configuring the Kernel.
```

5.3 Debug builds

Pass `DEBUG_BUILD=1` as an argument in the shell command prompt to create a debug build:

```
# setup the build environment
$ export SHELL=/bin/bash

$ MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment

# build qcom linux console image
$ DEBUG_BUILD=1 bitbake qcom-console-image
```

5.4 Update kernel command line

To update the kernel command line, modify the target-specific (`qcm6490.conf`) Yocto machine configuration variable `KERNEL_CMDLINE_EXTRA` in the corresponding machine configuration file.

For example, `./layers/meta-qcom-hwe/conf/machine/qcm6490.conf`.

To update the kernel command line, modify/append the following variable:

```
KERNEL_CMDLINE_EXTRA = "root=/dev/disk/by-partlabel/system rw rootwait
console=ttyMSM0,115200n8 pcie_pme=noms earlycon"
```

5.5 Platform support

Use the following procedure to update the DTB support in the kernel and select DTB on boot.

DTB build support in kernel

The device tree for a new platform can be integrated into the kernel build by updating the `Makefile`.

```
diff --git a/arch/arm64/boot/dts/qcom/Makefile b/arch/arm64/boot/dts/qcom/
Makefile
index 183aeba47193..a7815c774f7c 100644
--- a/arch/arm64/boot/dts/qcom/Makefile
+++ b/arch/arm64/boot/dts/qcom/Makefile
    dtb-$(CONFIG_ARCH_QCOM)      += qcm6490-addons-rb3gen2.dtb
+dtb-$(CONFIG_ARCH_QCOM)      += qcm6490-my-board.dtb
    dtb-$(CONFIG_ARCH_QCOM)      += qcm6490-rb3gen2.dtb
    dtb-$(CONFIG_ARCH_QCOM)      += qcs404-evb-1000.dtb
```

DTB inclusion in machine configuration

The Yocto machine configuration is also updated to include the corresponding device tree blob. For example, to add the device tree for QCM6490 machine support use the following file `conf/machine/qcm6490.conf`:

```
OUT_OF_KERNEL_DTSO - qcm6490.conf
# List of dtbs for corresponding supported qcm6490 platforms
KERNEL_DEVICETREE = " \
    qcom/qcm6490-my-board.dtb \
    qcom/qcm6490-addons-rb3gen2.dtb \
    qcom/qcm6490-my-board.dtb \
    "

# Do note additional list of dtbos to be overlaid on top of base kernel
devicetree
# Refer below how existing boards are managing it
# Format - KERNEL_TECH_DTBOS[<base-dtb-name>] = "<dtbo1 <dtbo2> ..."

KERNEL_TECH_DTBOS[qcm6490-addons-rb3gen2] = "qcm6490-graphics.dtbo qcm6490-
wlan-rb3gen2.dtbo qcm6490-display-rb3gen2.dtbo qcm6490-bt.dtbo"
```

DTB selection on boot

Custom DTB can be packaged as a part of the UKI image that can be updated in EFI to boot with the selected DTB.

Generate a UKI image using the ukify tool. The ukify tool is available as part of the Yocto build in the `tmp-glibc/sysroots-components/x86_64/systemd-boot-native/usr/bin/ukify` build directory.

Do the following to generate the UKI image:

```
# Note - ukify tool need python 3.10 version or above

$ ukify build --efi-arch=aa64 \
             --stub=<build-path>/tmp-glibc/deploy/images/qcm6490/
linuxaa64.efi.stub \
             --linux=<build-path>/tmp-glibc/deploy/images/qcm6490/Image \
             --initrd=<build-path>/tmp-glibc/deploy/images/qcm6490/initramfs-
qcom-image-qcm6490.cpio.gz \
             --cmdline="console=ttyMSM0,115200n8 earlycon
qcom_geni_serial.con_enabled=1 kernel.sched_pelt_multiplier=4
mem_sleep_default=s2idle" \
             --devicetree=<build-path>/tmp-glibc/deploy/images/qcm6490/
qcm6490-my-board.dtb \
             --output=./uki.efi
```

The ukify build command generates the `uki.efi` image with a custom board DTB.

Do the following to update the `uki.efi` image in the ESP partition `efi.bin` image.

```
# Following may need sudo privilege

# Take the yocto build generated efi.bin and mount it locally
$ mount <build-path>/tmp-glibc/deploy/images/qcm6490/efi.bin /mnt --options
rw

# Overwrite the uki.efi with one packaged above
$ cp uki.efi /mnt/EFI/Linux/uki.efi
$ umount /mnt

# now efi.bin carries packaged uki.efi which can be flashed to the target and
booted
# UEFI shall now pick the qcm6490-my-board.dtb that is part of uki.efi image

# reboot into fastboot and flash efi.bin
$ fastboot flash efi <build-path>/tmp-glibc/deploy/images/qcm6490/efi.bin
```

For more information on the device tree specification, see [The Devicetree Specification](#).

For Linux kernel documentation for device tree, see [Linux and the Devicetree](#).

5.6 Update ESP with images

Use the following steps to compile the systemd-boot boot manager and kernel images into a packaged UKI type-2 image file.

Yocto build for QCM6490 generates all the required images and package boot images as `efi.bin` that can be flashed in an EFI partition. `efi.bin` file consists of systemd-boot boot manager and kernel images, which are packaged as UKI type-2 image format.

Rebuild the EFI image after updating the kernel source, configuration, or DTS and flash the generated `efi.bin` to the EFI partition.

```
$ MACHINE=qcm6490 DISTRO=qcom-wayland source setup-environment

# build qcom linux console image
$ DEBUG_BUILD=1 bitbake qcom-console-image

# build images are produced in following directory
$ ls build-qcom-wayland/tmp-glibc/deploy/images/qcm6490/efi.bin
efi.bin

# reboot into fastboot
$ fastboot flash efi efi.bin
```

For more information on the UKI type-2 image format, see Type #2 [EFI Unified Kernel Images](#).

For information on ESP, see [Boot](#).

5.7 Customize initramfs

Modify the `PACKAGE_INSTALL` list in `meta-qcom-hwe/recipes-kernel/images/initramfs-qcom-image.bbappend` to update the initramfs package.

Use the following steps to update the initramfs package:

```
$ less meta-qcom-hwe/recipes-kernel/images/initramfs-qcom-image.bbappend

# Add additional packages needed as part of initrd
PACKAGE_INSTALL += " \
    e2fsprogs \
    e2fsprogs-e2fsck \
    e2fsprogs-mke2fs \
    e2fsprogs-resize2fs \
    e2fsprogs-tune2fs \
    ${VIRTUAL-RUNTIME_dev_manager} \
    os-release-initrd \
"
```

5.8 Add kernel module

Use the following procedure to compile out-of-tree kernel modules using the Yocto build system.

Sample Makefile for out-of-tree kernel driver:

```
all: modules
obj-m := hello.o

SRC := $(shell pwd)

modules:
    $(MAKE) -C $(KERNEL_SRC) M=$(SRC) modules $(KBUILD_OPTIONS)

modules_install:
    $(MAKE) -C $(KERNEL_SRC) M=$(SRC) modules_install
```

These kernel modules are integrated to the Yocto build using recipes that use the standard Yocto module class.

```
DESCRIPTION = "${SUMMARY}"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/${
{LICENSE}};md5=801f80980d171dd6425610833a22dbe6"

inherit module

SRC_URI += "file://Makefile \
            file://hello.c \
            file://COPYING \
            "
S = "${WORKDIR}"

EXTRA_OEMAKE += "MACHINE='${MACHINE}'"
MAKE_TARGETS = "modules"
MODULES_INSTALL_TARGET = "modules_install"

# Kernel module to be autoloaded
KERNEL_MODULE_AUTOLOAD += "hello"

# The inherit of module.bbclass will automatically name module packages with
# "kernel-module-" prefix as required by the oe-core build environment.

RPROVIDES_${PN} += "kernel-module-hello"
```

For more information on the out-of-tree module, see [Working with Out-of-Tree Modules](#).

5.9 Disable console log

Disable the kernel console logging in the following ways:

1. Update the kernel command line with `quiet`.
For more information on kernel parameters, see [The kernel's command-line parameters](#).
2. Disable `CONFIG_SERIAL_EARLYCON` and `CONFIG_SERIAL_MSM_CONSOLE` in the kernel configuration.
3. Add the `console=null` parameter to the kernel command line arguments.

5.10 Pinctrl configuration

The Pinctrl subsystem in the Linux kernel manages and configures pins used for general-purpose input/output (GPIO), interintegrated circuit (I2C), serial peripheral interface (SPI), and other hardware interfaces.

Pinctrl configurations, such as **pin muxing** and **pin groupings**, are managed in the device-specific pinctrl drivers, where the drivers list all the available pins and functions.

For example, the corresponding driver for QCM6490 is available in the `kernel-src/drivers/pinctrl/qcom/pinctrl-sc7280.c` file.

The following are the data objects:

Table 5-1 Pinctrl data objects

Variable	Description
<code>static const struct pinctrl_pin_desc sc7280_pins</code>	Enumerates all pins and their names.
<code>static const struct msm_pingroup sc7280_groups</code>	Defines the available muxed functions for the group of general-purpose input/output (GPIO) pins.
<code>enum sc7280_functions</code>	Lists all the available functions as enum values.

For more information on the supported functions of the respective SoC pinctrl binding documentation for QCM6490, see [pinctrl binding documentation](#).

Function selection

For a `sc7280_functions` data object, one or multiple GPIO pins are used as a function and must be registered to the device tree and passed to the right device node.

During system boot, the kernel pinctrl infrastructure registers the functions.

The following example shows the kernel configuration infrastructure:

```
tlmm: pinctrl@f100000 {
    compatible = "qcom,sc7280-pinctrl";
    :
```

```

:
:
:
qup_spi0_data_clk: qup-spi0-data-clk-state {
    pins = "gpio0", "gpio1", "gpio2";
    function = "qup00";
};

qup_spi0_cs: qup-spi0-cs-state {
    pins = "gpio3";
    function = "qup00";
};

qup_spil_data_clk: qup-spil-data-clk-state {
    pins = "gpio4", "gpio5", "gpio6";
    function = "qup01";
};

qup_spil_cs: qup-spil-cs-state {
    pins = "gpio7";
    function = "qup01";
};
:
:
:
:
};

spi0: spi@980000 {
    compatible = "qcom,geni-spi";
    reg = <0 0x00980000 0 0x4000>;
    clocks = <&gcc GCC_QUPV3_WRAP0_S0_CLK>;
    clock-names = "se";
    pinctrl-names = "default";
    pinctrl-0 = <&qup_spi0_data_clk>, <&qup_spi0_cs>;
    interrupts = <GIC_SPI 601 IRQ_TYPE_LEVEL_HIGH>;
    #address-cells = <1>;
    #size-cells = <0>;
    power-domains = <&rpmhpd SC7280_CX>;
    operating-points-v2 = <&qup_opp_table>;
    interconnects = <&clk_virt MASTER_QUP_CORE_0 0 &clk_virt
SLAVE_QUP_CORE_0 0>,
    <&gem_noc MASTER_APPSS_PROC 0 &cnoc2 SLAVE_QUP_0 0>;
    interconnect-names = "qup-core", "qup-config";
};

```

```

    dmas = <&gpi_dma0 0 0 QCOM_GPI_SPI>,
          <&gpi_dma0 1 0 QCOM_GPI_SPI>;
    dma-names = "tx", "rx";
    status = "disabled";
};

```

5.11 General GPIO usage

GPIO pin configuration requires the following two settings:

The settings define a GPIO pin state and make those pins available for any input/output activity.

- **Mux:** The mux setting requires selecting the function name that is mapped from the set of available functions in the target-specific pinctrl driver.

For more information on pinctrl, see [Pinctrl configuration](#).

- **Configuration:** The configuration aspect requires setting the drive-strength and bias property.

The following examples show how the two settings define the GPIO pin:

1. Define the pin configuration in the device tree using the following procedure:

```

bt_en: bt-en-state {
    pins = "gpio85";
    function = "gpio";
    output-low;
    bias-disable;
};

```

2. Use the following procedure to configure the device node or intellectual property (IP) block in the device tree:

```

bluetooth: bluetooth {
    compatible = "qcom,wcn6750-bt";
    pinctrl-names = "default";
    pinctrl-0 = <&bt_en>, <&sw_ctrl>;
    enable-gpios = <&tlmm 85 GPIO_ACTIVE_HIGH>;
    swctrl-gpios = <&tlmm 86 GPIO_ACTIVE_HIGH>;
    vddaon-supply = <&vreg_s7b_0p9>;
    vddbtcmx-supply = <&vreg_s7b_0p9>;
    vddrfacmn-supply = <&vreg_s7b_0p9>;
    vddrfa0p8-supply = <&vreg_s7b_0p9>;
    vddrfa1p7-supply = <&vreg_s1b_1p8>;
    vddrfa1p2-supply = <&vreg_s8b_1p2>;
    vddrfa2p2-supply = <&vreg_s1c_2p2>;
    vddasd-supply = <&vreg_l11c_2p8>;
    max-speed = <3200000>;
};

```

3. The driver code must use generic APIs to select and register their GPIO configurations within the pinctrl configurations.

The following is an example of available APIs:

```
devm_gpiod_get_optional(&serdev->dev, "enable", GPIOD_OUT_LOW);

/**
 * devm_gpiod_get_optional - Resource-managed gpiod_get_optional()
 * @dev: GPIO consumer
 * @con_id: function within the GPIO consumer
 * @flags: optional GPIO initialization flags
 *
 * Managed gpiod_get_optional(). GPIO descriptors returned from this
function
 * are automatically disposed on driver detach. See gpiod_get_optional()
for
 * detail
 */

/**
 * gpiod_get_value_cansleep() - return a gpio's value
 * @desc: gpio whose value will be returned
 *
 * Return the GPIO's logical value, i.e. taking the ACTIVE_LOW status into
 * account, or negative errno on failure.
 *
 * This function is to be called from contexts that can sleep.
gpiod_set_value_cansleep(qcdev->bt_en, 0);/**
 * gpiod_set_value_cansleep() - assign a gpio's value
 * @desc: gpio whose value will be assigned
 * @value: value to assign
 *
 * Set the logical value of the GPIO, i.e. taking its ACTIVE_LOW status into
 * account
 *
 * This function is to be called from contexts that can sleep.
```

GPIO as interrupt request (IRQ)

Use the following procedure to set the GPIO as an IRQ:

1. Configure the GPIO pin in the DTS file:
 - a. Set the properties and the function for the GPIO pin.
 - b. Set the pin to use GPIO 55 for `qup_se_13()` function with the following configurations:

```
qupv3_se3_rx: qupv3-se3-rx-state {
    pins = "gpio55";
    function = "qup03"; // To be taken from available from functions.
    drive-strength = <2>;
```

```
        bias-disable;
    };
```

2. Create a DT entry like the previous configuration for the device node where the GPIO must be set as an IRQ.

In the following example, the GPIO55 is configured as an IRQ with the parent being a top-level mode multiplexer (TLMM) and the level is set to high.

```
interrupts-extended = <&tlmm 55 IRQ_TYPE_LEVEL_HIGH>;
```

3. The driver must read the value and register it as an interrupt to the generic interrupt controller (GIC) using the `request_irq` API specifying the interrupt service register (ISR) and IRQ flags.

```
irq_no = platform_get_irq(pdev, 1);
```

5.12 ZRAM as swap

ZRAM is a compressed swap mechanism that creates a virtual block device in RAM. ZRAM is enabled as a module within the kernel defconfig.

ZRAM is configured in the Yocto build in the `recipes-extended/zram/zram/zram-swap-init-update` file.

To enable or configure ZRAM, do the following:

```
# check if zram module is loaded
$ lsmod | grep zram

# else load it
$ modprobe zram

# Configure /dev/zram0 size as per your RAM size
$ echo 128M > /sys/block/zram0/disksize

# activate swap
$ mkswap /dev/zram0
$ swapon /dev/zram0
```

To configure ZRAM as a swap device, see [zram: Compressed RAM-based block devices](#).

5.13 Extend memory map

To extend the memory map, adjust the addresses and sizes of memory regions in a DTSI file.

Use the following information to extend the carved out regions:

Add the carved out region in the `arch/arm64/boot/dts/qcom/<target>.dtsi` file in the **reserved-memory** node with the following syntax:

```
my_carveout_mem: my_carveout_mem@address {
    reg= <0x0 0xbase_address 0x0 0xsize>;
```

```

        no-map;
    }

```

For example:

```

my_carveout_mem: my_carveout_mem@d0800000 {
    reg= <0x0 0xd0800000 0x0 0x100000>;
    no-map;
}

```

Table 5-2 Syntax for carved out region

Variable	Description
my_carveout_mem@d0800000	Indicates the name of the device node. The convention mandates that you append the base address of the memory region to the name.
my_carveout_mem	Indicates the label assigned to this node that can be used to reference this node from within the other nodes in the device tree using phandles.
reg	Indicates the property, which is a 64-bit value that defines the base and size of the memory region.
no-map	Indicates that this region is carved out and the kernel should remove the mapping from its addressable range.

- NOTE**
1. None of the regions should overlap. In case a region size must be increased, all other subsequent regions must be shifted, and configured in the device tree to avoid overlapping.
 2. The kernel expects all memory region boundaries defined for kernel usage to be 1 MB.
 3. Existing carved out regions are protected by trusted firmware from kernel access. Decreasing their size or deleting them may result in an external abort, leading to a kernel crash.

Add CMA region

To add the CMA region in the `arch/arm64/boot/dts/qcom/<target>.dtsi` file under **reserved-memory** node, use the following syntax:

```

my_cma_mem: my_cma {
    compatible = "shared-dma-pool";
    alloc-ranges = <0x0 0x00000000 0x0 0xffffffff>;
    reusable;
    alignment = <0x0 0x400000>;
    size = <0x0 0x1400000>;
};

```

Table 5-3 Syntax for adding CMA region

Parameters	Description
<code>my_cma_mem</code>	Label for the CMA node and can be accessed as a phandle. The label <code>my_cma</code> is the name of the CMA region.
<code>shared-dma-pool</code>	Indicates that this region is a CMA area.
<code>alloc-ranges</code>	Indicates if the region should be within a certain memory limit, as some devices cannot access memory regions beyond the 32-bit address limit.
<code>reusable</code>	Indicates that the kernel can use the memory in this region when it is free.
<code>alignment</code>	Indicates any alignment requirements in this region.
<code>size</code>	Indicates the size of the region.
<code>reg</code>	An optional attribute that indicates the fixed region for memory allocation, in the absence of which memory is dynamically allocated at a random address.

5.14 Add custom CMA heaps

To create custom CMA regions using DMA-BUF heaps, use the existing DMA-BUF framework in the Linux kernel.

Qualcomm kernel exports the `cma_heap_add()` API for the custom DMA-BUF heaps.

```
/**
 * cma_heap_add - adds a CMA heap to dmabuf heaps
 * @cma:        pointer to the CMA pool to register the heap for
 * @data:        unused
 *
 * Returns 0 on success. Else, returns errno.
 */
```

```
int cma_heap_add(struct cma *cma, void *data);
```

The DMA-BUF heap exported to the user space, has the same name as the phandle of the CMA region in the device tree.

Steps to add a custom CMA heap:

1. To create a custom CMA region, see [Contiguous memory allocator \(CMA\)](#).
2. In the driver to which you are adding a custom heap:
 - a. Parse the device tree for the newly added CMA region.
 - b. Add a DMA-BUF heap associated with the driver.

```
int create_my_cma_heap(struct device *dev)
```

```

{
    int rc = 0, idx = 0;

    rc = of_reserved_mem_device_init_by_idx(dev, dev->of_node, 0); //
    Parse the devicetree for the cma region

    if (rc) {
        pr_err("No reserved DMA memory, ret=%d\n", rc);
        rc = -EINVAL;
        goto err;
    }

    rc = cma_heap_add(dev->cma_area, NULL); // Add a dmabuf heap
    associated with the cma region

    if (rc) {
        pr_err("cma_heap_add failed, ret=%d\n", rc);
        rc = -EINVAL;
        goto err;
    }

err:
    return rc;
}

```

NOTE CMA regions must be aligned to the 4 MB address base and size to support page migration. The migration happens at $2^{\text{pageblock_order}}$ pages at the page-block level. Page-block order is 10 in Qualcomm Linux.

5.15 Scheduler

The Qualcomm Linux kernel supports the standard Linux scheduler solution. The kernel uses the scheduler to choose the right CPU for task placement based on the CPU energy consumption.

PELT multiplier

In the Linux kernel, the PELT half-life multiplier influences how quickly the system adapts to changing workloads and adjusts CPU frequencies based on historical utilization data. It is a configuration that speeds up the PELT clock and reduces the half-life time, resulting in faster system response.

PELT multiplier is configured through the kernel command line argument:

```
kernel.sched_pelt_multiplier=[1, 2, 4] Default value: 1 (half life
32 msec), 2 (half life 16msec), 4 (half life 8 msec)]
```

Utilization clamping

Utilization clamping is a scheduler feature that allows user space to manage the performance requirements of tasks.

Clamping can be done in the following two ways:

- **UCLAMP_MIN:** If it is set to any value > 0 , the task demand is always greater than or equal to this value. If the actual task demand is more than this value, then the actual demand signal is used, but if the actual task demand is lower than this value, then **UCLAMP_MIN** is reported as the task demand.
- **UCLAMP_MAX:** If it is set to any value > 0 , the task demand is always less than or equal to this value. If the actual task demand is less than this value, then the actual demand signal is used, but if actual task demand is above this value, then **UCLAMP_MAX** is reported as task demand.

Note: **UCLAMP_MAX** $>$ **UCLAMP_MIN**

- **UCLAMP** can be used to influence the scheduler's task placement decisions.

On a heterogeneous system, the scheduler uses the task demand or utilization signal (PELT signal) to classify a task as small or big. Based on the input from the task classification, the scheduler selects small (less computing power) or big (more computing power) CPU cores for task placement with a probable impact on power consumption.

For example, nonimportant (trivial/background/housekeeping) tasks can be clamped to lower values (lower **UCLAMP_MAX**), whereby influencing the scheduler to place the tasks on the little cluster. Similarly, important/foreground/active tasks can be clamped to higher values (higher **UCLAMP_MIN**) whereby influencing the scheduler to place the tasks on the big cluster.

- **UCLAMP** also supports influencing the frequency guidance by the scheduler.

The task that needs a quick frequency ramp, can be clamped to higher demand (**UCLAMP_MIN**), which helps to ramp up the frequency of the cluster to meet the performance requirements of the task.

For more information on the interface and configuration, see [Utilization Clamping](#).

5.16 Change default CPU frequency governor

The CPU frequency governor can be changed at runtime or set statically during build compilation.

1. Kconfig configuration option: To set the CPU frequency governor, enable the corresponding driver in the kernel defconfiguration file.

To set **PERFORMANCE** governor as the default CPU frequency governor, set **CONFIG_CPU_FREQ_DEFAULT_GOV_PERFORMANCE=y** in the defconfiguration file.

2. Kernel command line option: To override the kernel configuration option, add a **cpufreq.default_governor=performance** parameter to the kernel command line in the **meta-qcom-hwe/conf/machine/qcm6490.conf** file to set the appropriate CPU frequency governor.

5.17 Customize cache and memory DVFS

The mapping between **CPU frequency** and **L3/DDR frequency** is adjusted based on the power or performance requirements.

In DTSI, for each CPUx node, there is an **operating-points-v2 = <cpux_opp_table>** entry. The **cpux_opp_table** holds a static mapping between CPU, L3, and DDR frequencies.

For example:

```
cpu0_opp_300mhz: opp-300000000 {
    opp-hz = /bits/ 64 <300000000>;
    opp-peak-kBps = <800000 9600000>;
};
```

When CPU 0 operates at 300 MHz, it votes for 9600000 to L3, which translates to 300,000 Hz (9600000 / w) L3 frequency. If the vote is for 800,000 Hz to DDR, this results in 200,000 Hz (800000 / w) DDR frequency.

In the equation, 'w' represents how many bytes can be written in a single cycle:

- For DDR, 'w' is 4 (each channel performs two transactions per cycle, with each transaction being 2 bytes).
- For L3, 'w' is 32 (one transaction per cycle at 32 bytes per transaction).

NOTE These values are set per channel for DDR, and the mapping relates CPU frequency to memory controller (MC) channel bandwidth. Adjusting this map table can impact power and performance characteristics.

For more information on the operating performance points (OPP) framework and syntax, see [Generic OPP \(Operating Performance Points\) Bindings](#).

5.18 Post boot settings

Post boot settings comprise completing initial setup, configuring memory, and CPU frequency parameters. These steps ensure proper configuration and functionality after the system boots up.

Post boot framework

A post boot script file is used to configure system parameters on the Qualcomm Linux distribution.

Postboot settings are managed through a *systemd* service file.

For more information on systemd, see [systemd\(1\) - Linux manual page](#).

What is a systemd service?

A systemd service is a background process that starts or stops based on specific conditions.

A *systemd* service file is written to allow systemd to parse, understand, and execute as instructed.

For more information on the background process, see [background process](#).

Basic structure of systemd service file

Use the following procedure to modify the behavior of systemd based on the requirement:

The following is the example of the systemd service file:

```
[Unit]
SourcePath=/etc/initscripts/log_restrict.sh
Description=QTI logging service
```

```
[Service]
ExecStart=/etc/initscripts/post_boot.sh

[Install]
WantedBy=multi-user.target
```

The [Unit] section

In systemd, a unit refers to any resource that the system knows how to operate on and manage. It encompasses services, sockets, devices, mount points, and groups of externally created processes.

Each unit is defined using a configuration file called a `unit` file, which contains metadata and configuration details. The `[Unit]` section within a unit file provides information about the description and relationships with other units.

The `unit` section has the following fields:

Table 5-4 Supported fields for [Unit] section

Fields	Description
Description	Human-readable title of the systemd service.
After	Set dependency on a service. For example, if you are configuring a WCN3960 web server, you want the server to start after the network is online.
Before	Start current service before specified service. In this example, the WCN3960 web server is running before the service for the next cloud is started because the next cloud server depends on the WCN3960 web server.

The [Service] section

The `Service` section contains details of the execution and termination of service.

The `Service` section has the following fields:

Table 5-5 Supported fields for [Service] section

Fields	Description
ExecStart	The command that must be executed when the service starts. For example, a web server service to start.
ExecReload	The optional field that it specifies how a service is restarted. For services that perform disk I/O, it is recommended to kill them and restart the service. Use the <code>ExecReload</code> field if you want to have a specific restart mechanism.
Type	Indicates the start-up type of a process for a given systemd service. The options available are <code>simple</code> , <code>exec</code> , <code>forking</code> , <code>oneshot</code> , <code>dbus</code> , <code>notify</code> , and <code>idle</code> .

For more information, see [Options](#).

The [Install] section

The `Install` section handles the installation of a `systemd` service or unit file. This is used when you run either `systemctl enable` and `systemctl disable` command for enabling or disabling a service.

The `Install` section has the following fields:

- **WantedBy:** Similar to the `After` and `Before` fields of the [The \[Unit\] section](#). However, it is used to specify `systemd`-equivalent **run levels**.
 The `default.target` is when all the system initialization is complete and when asked to log in. Most user-facing services (like `WCN3960`, `cron`, `GNOME-stuff`) use this target.
 The `shutdown.target` is to run the service just before the device shuts down.
 The `multi-user.target` is to run the service at the time of system boot, as the root user.
 It can be either a target or a service, for example `network.target`.
- **RequiredBy:** This field is similar to `WantedBy`. However, the field specifies **hard dependencies**. If a dependency fails, the service also fails.

Post boot script

The post boot script file is in the core recipe of the `meta-qcom-hwe` layer.

The post boot script to be run as a part of the previous `systemd` service. All post boot settings can be hosted in a `meta-qcom-hwe/recipes-core/initscripts/files/post_boot.sh` script.

```
#!/bin/sh
# Copyright (c) 2023 Qualcomm Innovation Center, Inc. All rights reserved.
# SPDX-License-Identifier: BSD-3-Clause-Clear

# Apply post boot settings
```

Recipe for postboot systemd service

Use the `initscripts_1.0.bbappend` file to understand the recipe configuration procedure to install `post_boot` script.

The recipe to install the postboot bash script in `/etc/init.d` is in the `meta-qcom-hwe/recipes-core/initscripts/initscripts_1.0.bbappend` file.

```
# postboot

inherit systemd externalsrc

FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SRC_URI:append = " \
    file://post_boot.sh \
    file://logging-restrictions.sh \
```

```

        file://log-restrict.service \
        file://post-boot.service \
    "

do_install:append() {
    # postboot
    install -m 0755 ${WORKDIR}/post_boot.sh ${D}${sysconfdir}/initscripts/
post_boot.sh
    install -m 0644 ${WORKDIR}/post-boot.service -D ${D}${systemd_unitdir}/
system/post-boot.service
    ln -sf ${systemd_unitdir}/system/post-boot.service ${D}${systemd_unitdir}/
system/multi-user.target.wants/post-boot.service
}

S = "${WORKDIR}"

INITSCRIPT_PACKAGES += "${PN}-post-boot"
INITSCRIPT_NAME:${PN}-post-boot = "post_boot.sh"

PACKAGES += "${PN}-post-boot"
FILES:${PN}-post-boot += "${systemd_unitdir}/system/post-boot.service $
${systemd_unitdir}/system/multi-user.target.wants/post-boot.service $
${sysconfdir}/initscripts/post_boot.sh"

```

Scheduler DCVS settings

Scheduler DCVS settings are defined in the `post_boot` script file.

All post boot settings are present in the `meta-qcom-hwe/recipes-core/initscripts/files/post_boot.sh` script.

```

#!/bin/sh
# Copyright (c) 2023 Qualcomm Innovation Center, Inc. All rights reserved.
# SPDX-License-Identifier: BSD-3-Clause-Clear

```

6 Debug

Use the following methods to understand the high-level categorization of ways to debug the kernel, and report issues identified with the inherent debug features in the Linux kernel.

For more information on debug, see [Qualcomm Linux Debug Guide](#).

Serial console

Standard boot and kernel logs are accessible using a serial console. The serial console enables live debugging on a live target. Qualcomm kernel enables the `CONFIG_SERIAL_QCOM_GENI` driver to support UART and console.

Add the following line to the kernel command line parameter:

```
console=ttyMSM0,115200n8

# as in following line in meta-qcom-hwe/conf/machine/qcm6490.conf
KERNEL_CMDLINE_EXTRA ?= "root=/dev/disk/by-partlabel/system rw rootwait
console=ttyMSM0,115200n8 pcie_pme=noms earlycon"
```

When you rebuild and load the kernel:

- Connect a Micro USB to USB-A type cable as follows:
 - Device > Micro USB====USB-TypeA > Host machine
- To connect to a device serial port on the host, use a serial console client on a host machine.
- You should be able to see serial console logs and other kernel logs on serial client.

For more information on the serial console, see [Linux Serial Console](#).

Configure console log level

Configure the kernel console log level for extensive or minimal logging to satisfy debug or release software versions.

Use `/proc/sys/kernel/printk` to configure log level to control log messages appearing on the console. Set the log level from 1 to 7.

When you set the log level value to 1, it filters the log to the lowest level. When the log level value is set to 1, only `pr_emerg/KERN_EMERG` `printk` logs are printed.

When you set the log level value to 7, it enables the highest log level `pr_info/KERN_INFO` and prints all `printk` logs to the console.

```
$ echo "1" > /proc/sys/kernel/printk
```

Display kernel logs

Run either of the following commands to display the kernel logs:

```
$ dmesg
# or
$ cat /proc/kmsg
```

Display kernel logs since boot up

Run either of the following commands to display the kernel logs since boot up:

```
$ cat /var/log/messages
or
$ cat /var/log/kern.log
```

Debugging with `printk`

Debugging with `printk` is a technique in the Linux kernel for printing messages and tracing.

The wrappers around `printk` defined in `include/linux/printk.h`, supports adding a log level to your log statements.

For example,

```
pr_emerg("At line %d: Func: %s\n", __LINE__, __func__);
pr_info("At line");
```

For more information, see [Message logging with `printk`](#).

Using the log levels

You can print messages using any other log levels defined in `/linux/printk.h`. The console logs are controlled using the log level used in `printk` and the log level chosen on the device.

Log level should be chosen with each case according to your use case. You can run into a lot of logs if a lower log level is chosen in a frequently called function.

The following example shows the logs for kernel print levels:

```
pr_info("At func %s\n", __func__);
pr_notice("At func %s\n", __func__);
pr_warn("At func %s\n", __func__);
pr_err("At func %s\n", __func__);
pr_crit("At func %s\n", __func__);
pr_alert("At func %s\n", __func__);
pr_emerg("At func %s\n", __func__);
```

6.1 Enable SSH

Enable secure shell (SSH) in permissive mode to securely access your host device. For instructions, see [How to SSH?](#).

6.2 Retrieve information from device

You can retrieve the information from your device to debug the kernel issues.

The following is the list of kernel commands:

Table 6-1 List of kernel commands

Commands	Description
<code>\$ zcat /proc/config.gz</code>	Kernel configuration
<code>\$ cat /proc/cmdline</code>	Kernel command line parameters
<code>\$ cat /proc/version</code>	Kernel version
<code>\$ ls /proc/device-tree</code>	Device-tree configuration on device
<code>\$ lsmod</code>	Loaded modules
<code>\$ cat /proc/interrupts</code>	Status of interrupts
<code>\$ cat /sys/devices/system/cpu/cpufreq/policyX/scaling_available_frequencies</code>	Available CPU frequencies where X = cluster
<code>\$ cat /sys/kernel/debug/qcom_socinfo/chip_id</code>	chip_id using Enable debugfs .

For more information on `procfs`, see [The /proc Filesystem](#).

6.3 Enable debugfs

Debugfs is a file system that allows kernel developers to provide kernel information to the user space.

Debugfs is used to access kernel data structures and variables, trace events, debug messages, and statistics.

- Ensure that your kernel configuration has `CONFIG_DEBUG_FS` set (enabled by default).
- If not, you can enable it in your kernel configuration using `menuconfig`.
- Mount debugfs if not already mounted.

```
$ mount -t debugfs none /sys/kernel/debug
```

6.4 Debugging kernel and modules through KGDB

Kernel GNU debugger (KGDB) provides live debugging support for the kernel on a target.

You can configure KGDB to debug core kernel and module issues. KGDB is a source-level debugger used with GDB to debug the Linux kernel.

For information on KGDB kernel debugging, see [Debugging kernel and modules via gdb](#).

Use KGDB with serial COM port

Enable the following kernel drivers in the kernel configuration file to support KGDB with serial COM port:

```
CONFIG_FRAME_POINTER
CONFIG_KGDB
CONFIG_KGDB_SERIAL_CONSOLE
CONFIG_HAVE_ARCH_KGDB
CONFIG_CONSOLE_POLL
CONFIG_MAGIC_SYSRQ
```

Update the command line for debug:

```
# Append KERNEL_CMDLINE_EXTRA in meta-qcom-hwe/conf/machine/qcm6490.conf
with following string
```

```
"kgdboc=ttyMSM0,115200n8 kgdbwait nokaslr"
```

```
# If waiting during the boot of kernel for gdb connection is not desired,
# the parameter 'kgdbwait' can be skipped.
```

Disable Watchdog

- Disable the watchdog when debugging using KGDB. The SoC resets when the watchdog is enabled.
- Add the following kernel command line parameter to disable the watchdog:

```
echo 1 > /sys/bus/platform/devices/hypervisor:qcom,gh-watchdog/disable
```

Ways to enter debug mode

Method 1 - Use kgdbwait:

If `kgdbwait` is passed as the command line parameter, the kernel pauses the execution during boot and enters Debug mode and waits for the connection from remote GDB, with the following message:

```
[ 0.239669] printk: console [ttyMSM0] enabled
[ 1.535669] random: fast init done
[ 1.541411] KGDB: Registered I/O driver kgdboc
[ 2.224804] KGDB: Waiting for connection from remote gdb...
```

Disable the watchdog from the configuration.

Method 2 - Use Magic SysRq:

To enter the Debug mode, run `SysRq-G` in the shell, and execute the following command:

```
echo g > /proc/sysrq-trigger
```

Method 3 - Whenever you want to hit a KGDB breakpoint in the source code. Add the following code snippet to the intended driver file.

```
#include <linux/kgdb.h>    (you may need to include this file )
kgdb_breakpoint(); //call this for adding a break point at compile time
```

Method 4 - Triggering panic: The kernel automatically enters Debug mode when an exception occurs. To trigger a crash, use Magic SysRq feature.

```
echo c > /proc/sysrq-trigger
```

Connecting to the target (SoC) through GDB over COM Port

Install GDB `gdb-multiarch`: `mingw64\bin\gdb-multiarch.exe`.

- GDB for Windows: Install Mingw-w64 through [MSYS2](#).
- GDB for Linux: Install GDB using [How to Install GDB](#).
- All other connections to a serial port must be closed, while using COM Port with GDB to connect to the SoC.
- On a Windows host - COM Port number must be set to greater than 16. You can get the port number from the device manager.
- On a Windows host - COM Port should be passed as `\\.com<#>`
- In the following example, connect from a serial port on a Windows host. The serial port number is COM17. GDB is disconnected from COM Port before connecting PuTTY again. To attach and de-attach GDB commands, run the following commands:

```
gdb-multiarch.exe -b 115200 <path_to_vmlinux>
(gdb) target remote \\.com17
Remote debugging using \\.com17
warning: multi-threaded target stopped without sending a thread-id,
using first non-exited thread
[Switching to Thread -2]
arch_kgdb_breakpoint () at arch/arm64/include/asm/kgdb.h:21
21      arch/arm64/include/asm/kgdb.h: No such file or directory.
```

Use the following commands to set the breakpoint to view the breakpoints, or to view the CPU registers:

```
# backtrace:
> bt

# set a break point:
break <function_name>
or
break <filename>: line no
```

```
# View all current break points:
info break

# View CPU registers:
info reg

# step over:
s

# continue execution:
c
```

Using KGDB to debug kernel modules

KGDB is a kernel patch that enables source-level debugging of a running Linux kernel using the GDB interface.

You must have the `.ko` files and location of the `.text` kernel module files.

To get the address of the `.text` section of the loaded module, run:

```
# from live target
# cat /sys/module/<module>/sections/.text
```

To add a module symbol file, run:

```
(gdb) add-symbol-file <path_to_.ko> <.text_addr>
```

For example, `add-symbol-file /sys/modules/linux/linux.ko 0xc0ae22d0`.

6.5 Function trace

The `ftrace` provides tracing utilities that do system-wide profiling and tracing at runtime.

For more information on `Ftrace`, see [Function trace](#).

6.5.1 Enable memory-mapped input/output traces

The memory mapped I/O (MMIO) traces in the Linux kernel provide information to understand how drivers interact with MMIO devices and their associated hardware states.

Generic MMIO traces use `__raw_{read,write}{b,l,w,q}` accessors to read/write from/to memory-mapped registers.

In the following cases, the device hangs or some undefined behavior leads to device crashes:

Table 6-2 Reasons of device crashes

Reasons	Explanation
Unclocked access	If the access to the register space is unlocked, the device crashes.
Protected register space	If the register space is protected, and not set for access to the nonsecure world exceptions happen. For example, only exception level (EL3) access is allowed and any EL2/EL1 access is forbidden.
xPU control	xPU memory protection units control access to certain memory or register regions for specific clients.

The previous scenarios result in instant reboot/synchronous errors (SErrors)/network on chip (NoC), or interconnect hang. The `CONFIG_TRACE_MMIO_ACCESS` provides ftrace trace events to log such MMIO register accesses, which offer early enablement of trace events, filtering capability, and dumping the ftrace logs on console.

The following is the sample output from the trace buffer when the MMIO traces are enabled:

```
# List all rwmio trace events
$ cat /sys/kernel/tracing/available_events | grep rwmio

# Enable all rwmio trace events
$ cat /sys/kernel/tracing/available_events | grep rwmio >> /sys/kernel/
tracing/set_event

# List all traces
$ cat /sys/kernel/tracing/trace
rwmio_read: gic_peek_irq+0xd0/0xd8 readl addr=0xffff800010040104
rwmio_write: gic_poke_irq+0xe4/0xf0 writel addr=0xffff800010040184
rwmio_read: gic_do_wait_for_rwp+0x54/0x90 readl addr=0xffff800010040000
rwmio_write: gic_set_affinity+0x1bc/0x1e8 writeq addr=0xffff800010046130
```

6.5.2 Kprobes

Kprobes enables you to break into any kernel routine and collect debugging and performance information nondisruptively.

Kernel code address is trapped using a handler routine to invoke when the breakpoint is hit. Kprobes are useful during scheduler debug when you generate traces at different scheduling events to understand the system behavior.

For more information, see [Kernel Probes](#).

6.6 Troubleshoot kernel problems

Use the following methods to troubleshoot the kernel issues:

Boot failure due to incorrect or no DTB picked

When troubleshooting boot failure due to DTB load issues, verify authentication and DTB availability, and follow the steps to fix the problem.

The following scenarios lead to boot failure:

- **Authentication failure**

The following example shows the logs for failure due to incorrect authentication:

```
Platform Subtype : 0
DtPlatformLoadDtb qcm6490-rb3gen2.dtb is loaded
Platform Subtype : -2090817768
DtPlatformLoadSign qcm6490-rb3gen2.sgn is loaded
failed to authenticate image !
```

- **Failure due to unavailability of DTB**

The following example shows the logs for failure due to the missing DTB:

```
DtPlatformLoadDtb qcm6490-rb3gen2.dtb is loading failed with Status = E
DtPlatformDxeEntryPoint: no DTB blob could be loaded, defaulting to ACPI
(Status == Not Found)
```

Use the following steps to fix the unavailability of DTB issues:

- Verify if the corresponding DTB file is a part of a packaged `efi.bin` file.
- Mount `efi.bin` file locally on the host development machine to verify the following:

```
$ mount -t vfat efi.bin /mnt/

$ ls -lR /mnt/
```

Serial console not working

To troubleshoot serial console log failure, enable specific drivers in the kernel configuration and add relevant parameters to the kernel boot arguments.

- Enable the following driver in the kernel configuration file:

- `CONFIG_SERIAL_QCOM_GENI=y`
- `CONFIG_SERIAL_QCOM_GENI_CONSOLE=y`

- Add the following parameter to the kernel boot arguments:

```
console=ttyMSM0,115200n8
```

- To get early boot messages from the kernel, add the following parameter to the kernel boot arguments:

```
earlycon
```

Remoteproc not coming up

To troubleshoot remoteproc failure, capture kernel logs, use matching firmware signature files and check firmware locations in the device.

- Ensure that all the matching subsystem images are properly flashed without any errors.
- Verify if there are any errors in the kernel log.

The following example shows a sample kernel log:

```
0x000000000A27652C | 5198.790423: qcom_q6v5_pas 3000000.remoteproc:
fatal error received: err_inject_crash.c:413:Crash injected via Diag
0x000000000A276689 | 5198.801061: remoteproc remoteproc2: crash
detected in 3000000.remoteproc: type fatal error
0x000000000A2767A1 | 5198.809602: remoteproc remoteproc2: handling
crash #1 in 3000000.remoteproc
0x000000000A27688E | 5198.816837: remoteproc remoteproc2: recovering
3000000.remoteproc
0x000000000A276971 | 5198.823784: qcom_q6v5_pas 8a00000.remoteproc:
subsystem event rejected
```

- Disable the following subsystem restart feature to ensure that the remoteproc crash signature is visible in the kernel log:

```
echo disabled > /sys/kernel/debug/remoteproc/remoteprocN/recovery
```

- Confirm if all the necessary firmware files are present in the `/lib/firmware/qcom/qcm6490` in `rootfs` file system.

Debug configurations or symbols not reflecting in images

To debug kernel configuration change issues where configurations or symbols are not reflecting in the images, follow these steps:

- Add the debug configuration driver to the `arch/arm64/configs/qcom_debug.config` file.
- Export the macro `DEBUG_BUILD=1` before executing the `bitbake` command.

System memory is too low

Use the procedure mentioned in the following link to debug cases where a system runs out of free memory.

For more information on memory, see [Out of memory](#).

7 References

Related documents

Title	Number
Qualcomm Technologies, Inc.	
Qualcomm Linux Debug Guide	80-70014-12
Qualcomm Linux Build Guide	80-70014-254
Qualcomm Linux Boot Guide	80-70014-4
Qualcomm Linux Yocto Guide	80-70014-27
Resources	
Linux kernel-EFI	https://docs.kernel.org/admin-guide/efi-stub.html
Linux kernel API doc	https://www.kernel.org/doc/html/latest/driver-api/gpio/index.html
Memory	https://docs.kernel.org/core-api/index.html#memory-management
Scheduler	https://docs.kernel.org/scheduler/sched-energy.html
CPU topology	https://docs.kernel.org/scheduler/sched-capacity.html
DT configuration setting	https://www.kernel.org/doc/Documentation/devicetree/bindings/gpio/gpio.txt
DT pin configuration	https://www.kernel.org/doc/Documentation/devicetree/bindings/pinctrl/pinctrl-node.yaml
PELT	https://lwn.net/Articles/531853/
Schedutil governor	https://docs.kernel.org/scheduler/schedutil.html
UCLAMP	https://docs.kernel.org/scheduler/sched-util-clamp.html
DVFS	https://www.kernel.org/doc/Documentation/cpu-freq/governors.txt
Remoteproc framework	https://www.kernel.org/doc/Documentation/remoteproc.txt
Serial console	https://docs.kernel.org/admin-guide/serial-console.html
Debugging with printk	https://www.kernel.org/doc/html/next/core-api/printk-basics.html
File systems	https://docs.kernel.org/filesystems/proc.html
Debugfs	https://www.kernel.org/doc/html/next/admin-guide/dynamic-debug-howto.html
GDB	https://docs.kernel.org/dev-tools/gdb-kernel-debugging.html
Debug memory	https://www.kernel.org/doc/html/v5.0/vm/page_owner.html
Slub Debug	https://www.kernel.org/doc/Documentation/vm/slub.txt
Kernel configuration	https://docs.yoctoproject.org/4.3.1/kernel-dev/common.html#configuring-the-kernel

Acronyms and terms

Acronym or term	Definition
ASMP	Asymmetric multiprocessing
APSS	Applications processor subsystem
aDSP	Audio digital signal processor
CMA	Contiguous memory allocator
cDSP	Compute digital signal processor
COFF	Common object file format
DDR	Double data rate
DMIPS	Dhrystone million instructions per second
DTSI	Device tree source inclusion
DVFS	Dynamic voltage and frequency scaling
EAS	Energy-aware scheduling
EFI	Extensible firmware interface
ELF	Executable and linking format
EM	Energy model
ESP	EFI system partition
FAT	File allocation table
GIC	Generic interrupt controller
GPIO	General-purpose input/output
I2C	Interintegrated circuit
IRQ	Interrupt request
IPC	Interprocess communication
ISR	Interrupt service routine
L3	Level 3 cache
LLCC	Last level cache controller
LPASS	Low-power audio subsystem
LTS	Long-term support
MPSS	Modem peripheral subsystem software
MTP	Mobile test platform
OS	Operating system
PAS	Peripheral authentication service
PBL	Primary boot loader
PE	Portable executable file format
PELT	Per entity load tracking
PD	Protection domain
PIL	Peripheral image loader
QPST	Qualcomm Product Support Tool

Acronym or term	Definition
QXDM	Qualcomm extensible diagnostic monitor
RPE	Reset, power, error
SLPI	Sensor low-power island
SMD	Shared memory driver
SMP2P	Shared memory point to point
SoC	System on chip
SPI	Serial peripheral interface
SSR	Subsystem restart
TLMM	Top-level mode multiplexer
UCLAMP	Utilization clamping
UEFI	Unified extensible firmware interface
UKI	Unified kernel image
URI	Uniform resource identifier
WCNSS	Wireless connect subsystem
WDOG	Watchdog service
WLAN	Wireless local area network
WPSS	Wireless processor subsystem
XBL	eXtensible boot loader

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“Qualcomm Technologies”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “Qualcomm Internal Use Only”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com or the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.